

MEDIATEK

MT8391 MT8371 Android Co-processor SCP Developer Guide

Version: 1.0
Release date: 2024-10-14

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-10-14	Hailong Fan	Official release

Table of Contents

Version History	2
Table of Contents	3
List of Figures	5
List of Tables	5
1 System Overview	6
1.1 Hardware Architecture	6
1.2 Software Architecture	7
1.3 Summary	8
2 Abbreviations	9
3 Source Tree	10
3.1 Little Kernel Bootloader	10
3.2 Linux Kernel Driver	10
3.3 FreeRTOS Tree	10
4 Build System	11
4.1 Configuration Files	11
4.1.1 Little Kernel Bootloader	11
4.1.2 Linux Kernel	11
4.1.3 FreeRTOS	11
4.2 Build Commands	11
4.2.1 Standalone	11
4.2.2 With AOSP Hierarchy	12
4.3 Image Layout	12
5 Boot Sequence	13
6 Inter Processor Interrupt (IPI)	14
6.1 Usage on the SCP Side	14
6.2 Usage on the Kernel Side	19
7 DRAM Access Procedure	21
7.1 Reserving Memory in Linux Kernel	21
7.1.1 Register an User to Shared Memory	22
7.1.2 Extend the Shared Memory Size	23
7.2 Get Reserved Memory by ID	23
7.3 Get Reserved DRAM Region on SCP Side	23
7.4 Remap DRAM Address from AP View to SCP View	24
7.5 Request System Bus and DRAM	25
8 Drivers Guide	26
8.1 Driver Initiation	26
8.2 Add a New Driver	26
8.3 Interrupt	26
8.3.1 IRQ Registration	27
8.3.2 Enable IRQ	28
8.3.3 Disable IRQ	28
8.3.4 Wakeup Source Registration	29
8.4 Locks	29

8.5	DMA	30
8.6	Hardware Semaphore	31
8.7	GPIO & EINT	32
8.7.1	GPIO Usage	32
8.7.2	EINT Usage	33
9	Debug Methods	34
9.1	PRINTF_* Usage	34
9.2	Mobile Log	34
9.3	UART	35
9.4	ADB Logcat	35
9.5	Exception Log Analysis	36
9.5.1	Trace Buffer	38
9.6	Core Dump	38
9.6.1	LLDB Basic Commands	39
9.7	Performance Budget and Flight Record	42
9.7.1	Performance Budget	42
9.7.2	Flight Record	43
10	Address Sanitizer	49
10.1	ASAN Usage	49
10.2	How to Move Code to DRAM	51
11	Appendix	53
11.1	Common Issues and How to Fix	53
11.1.1	Malloc Fail	53
11.1.2	Unaligned Access	54
11.2	Code Size Limitation	54
11.2.1	Code Size Tool Usage	55
11.3	Scp_Region_Info Structure	56
11.4	SCP Recovery	56
11.4.1	Recovery Behavior	57
11.4.2	Recovery Notify Flow	57
12	Q&A List from Users	59
12.1	How to Enlarge DRAM Region Code	59
	Exhibit 1 Terms and Conditions	60

List of Figures

Figure 1-1. MT8391 SCP architecture	7
Figure 1-2. MT8391 SCP software architecture	8
Figure 5-1. SCP boot up flow overview	13
Figure 6-1. Tinsys common IPI architecture	14
Figure 7-1. DRAM access procedure	21
Figure 9-1. MTK Logger	34
Figure 9-2. UART setting	35
Figure 9-3. Disable SCP Mobile log	36
Figure 9-4. ADB logcat output	36
Figure 9-5. Performance budget log example	43
Figure 9-6. Flight record console mode example	45
Figure 9-7. Button instruction	45
Figure 9-8. Flight record GUI mode interface	46
Figure 9-9. Detail information	46
Figure 9-10. Task wakeup event	46
Figure 9-11. Example of queue event	47
Figure 9-12. Detail information of queue event	47
Figure 9-13. Example of SW timer	47
Figure 9-14. Example of critical section	47
Figure 9-15. Example of ISR	48
Figure 9-16. Example of OS Tick	48
Figure 11-1. SCP recovery behavior	57

List of Tables

Table 1-1. Hardware specification	6
Table 1-2. Architecture specification	8
Table 2-1. Abbreviations	9
Table 7-1. The standard DRAM access flow	21
Table 7-2. The default remapping rule table	24
Table 8-1. The EINT and GPIO mapping ball name	32
Table 8-2. The GPIO control register table	32
Table 9-1. PRINTF use scenario	34
Table 9-2. UART pin name	35
Table 9-3. Compiler options and definitions	42
Table 9-4. Compiler options	43
Table 9-5. Definitions of Flight Record	43
Table 10-1. Compiler options and definitions	49
Table 10-2. Exception type	49

1 System Overview

Sensor-hub Control Processor (SCP) is a sub-system that is designed to perform always-on tasks even when the system is in low power states, such as Voice Wakeup, and Sensor Hub. This chapter describes the HW and SW architecture of SCP.

1.1 Hardware Architecture

SCP consists of dedicated processor(s), SRAM, DMA, and peripherals, e.g., I2C, GPIO. An in-house DSP and new architecture have been introduced since MT8391 so that always-on applications could run with lower power and gain better performance.

- Processor: In-house MDSP RV55x1 and 2 hardware threads each core, using RISC-V architecture
 - A single precise floating point
 - Compressed instruction
 - DSP ISA for voice acceleration
 - Individual I cache 32K, and D cache 32K for each core
 - OPPs from 26 MHz to 416 MHz
- Memory: 384KB TCM
- DMA throughput:
 - Data transfer from TCM to DRAM: 63.8 MB/s
 - Data transfer from DRAM to TCM: 39.9 MB/s
- Peripherals:
 - I2C × 2
 - SPI × 2
 - UART × 1
- Details

Table 1-1. Hardware specification

Item		MT8391
Core		RV55 × 1 (2 HW threads per core)
Cache	L1\$	I\$/D\$: 32KB/32KB
	L2\$	NA
TCM	L1TCM	NA
	L2TCM	384KB
Peripherals		• I2C × 2 • SPI × 2 • UART × 1
DMA		1 channel
VoW I/F		NA
Operating Frequency		416M Hz@VLP 0.75V
Performance (CoreMark@Vmin)		TBD

Item	MT8391
Power Efficiency (CoreMark/mW)	TBD

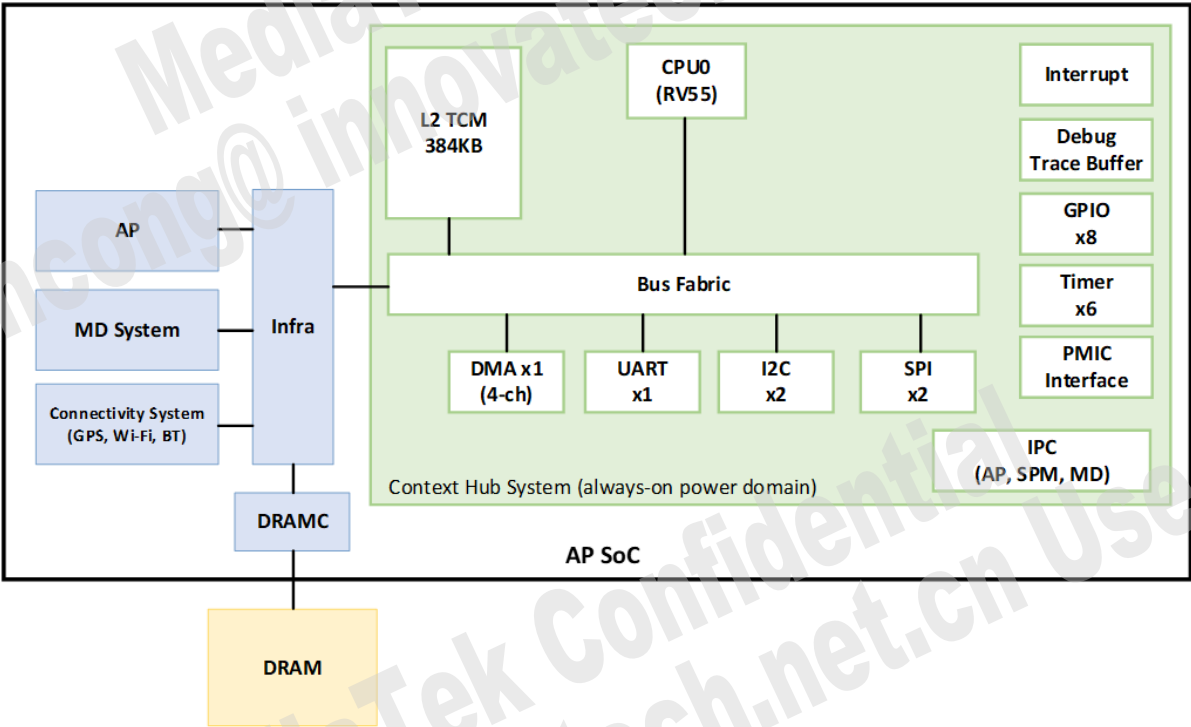


Figure 1-1. MT8391 SCP architecture

1.2 Software Architecture

SCP SW is based on AWS FreeRTOS v10.1.0.1 which is a real time kernel in MIT v2 license and supports multi-task, mutex, semaphore and software timer.

The SW package also includes additional middleware such as audio and sensor hub, but the details will not be discussed in this document.

SCP communicates with the Linux kernel via Inter Processor Interrupt (IPI) based on OpenAMP rpmsg/remoteproc framework. Similarly, other mechanisms are provided to enable co-work between AP and SCP:

- IPI: See [Chapter 6](#)
- HW semaphore: See [Section 8.6](#)
- Logger: See [Section 9.2](#)
- Recovery: See [Section 11.4](#)

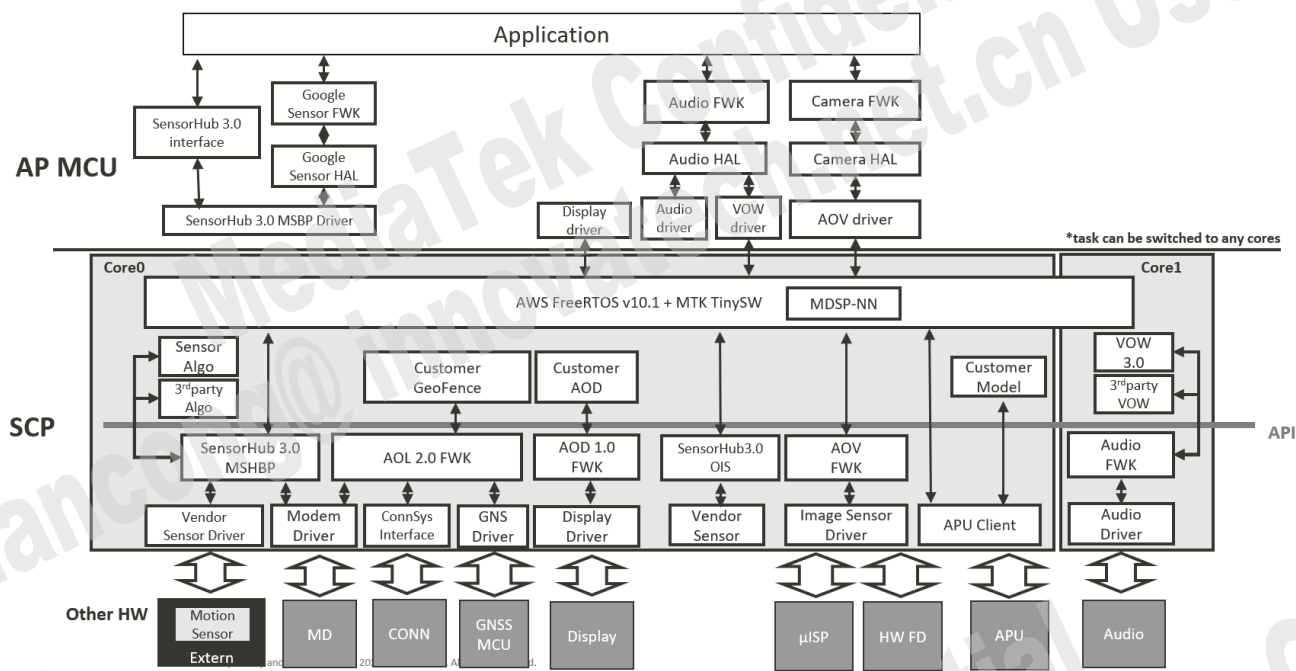


Figure 1-2. MT8391 SCP software architecture

1.3 Summary

Table 1-2. Architecture specification

Item	Value
Platform	mt8391
Project	\$PROJECT
Linux version	6.1
FreeRTOS version	10.1.0.1
ISA	RV55
L2 TCM	384KB

2 Abbreviations

Table 2-1. Abbreviations

Abbreviation	Explanation
API	Application Programming Interface
ASAN	Address Sanitizer
IPI	Inter Processor Interrupt
SCP	Sensor-hub Control Processor

3 Source Tree

SCP's source tree includes the bootloader (i.e., LK), Linux kernel, and FreeRTOS. See the following subsections.

3.1 Little Kernel Bootloader

`vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/common/scp`

3.2 Linux Kernel Driver

- SCP driver path
 - `kernel/kernel_device_modules-6.1/drivers/misc/mediatek/scp`
- SCP DTS path
 - `kernel/kernel_device_modules-6.1/arch/arm64/boot/dts/mediatek/mt8189.dts`

3.3 FreeRTOS Tree

- RTOS kernel
 - `vendor/mediatek/proprietary/tinysys/kernel/FreeRTOS_v10.1.0.1`
- Platform and peripheral drivers
 - `vendor/mediatek/proprietary/tinysys/scp`
 - `vendor/mediatek/proprietary/tinysys/common`
- Libraries
 - `vendor/mediatek/proprietary/tinysys/scp/middleware`
- Toolchain
 - `prebuilts/clang/md32rv/linux-x86/CodeLine_212`

4 Build System

Before digging into programming, developers might want to know how to compile with the basic SCP configuration. This chapter will show where and how configurations are arranged and how to produce an SCP image using commands.

4.1 Configuration Files

4.1.1 Little Kernel Bootloader

vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/mediatek/mt8189/rules.mk

- `MODULES_DEP += platform/$(PLATFORM)/common/scp/RV`

4.1.2 Linux Kernel

Kernel/kernel_device_modules-6.1/arch/arm64/configs/mgk_64_k61_defconfig

- Enable/disable SCP driver: `CONFIG_MTK_TINYSYS_SCP_SUPPORT`
- Features switch off, such as Voice Wakeup and Sensor Hub

4.1.3 FreeRTOS

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/platform.mk

- **Default** configurations of the platform
- Extra CFLAGS
- Extra LDFLAGS
- Driver/middleware C objects and the path included

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/\$PROJECT/project.mk

- Project-specific configuration
- **Customize** the project by overriding options in platform.mk

4.2 Build Commands

There are several ways to build SCP images. Two of them are listed in the ALPS SW packages.

4.2.1 Standalone

The SCP firmware can be constructed rapidly using this method, particularly beneficial during development.

- The output path: `./tinysys_out`
- SYNOPSIS: `PROJECT=XXX TARGET_BOARD_PLATFORM=XXX BUILD_TYPE=[release, debug] make`
- For example:

```
$ cd vendor/mediatek/proprietary/tinysys/scp
$ PROJECT=fpga8189_64 TARGET_BOARD_PLATFORM=mt8189 BUILD_TYPE=debug make -j24
```

4.2.2 With AOSP Hierarchy

- The output path: out/target/product/\$PROJECT/obj/TINYSYS_OBJ/tinysys-scp_intermediates/RV55_A/scp
- For example:

```
$ make tinysys-scp -j24
$ mmm vendor/mediatek/proprietary/tinysys/scp -j24
$ cd vendor/mediatek/proprietary/tinysys/scp && mm -j24
$ mmm vendor/mediatek/proprietary/tinysys/scp:tinysys-scp -j24
```

4.3 Image Layout

- Partition in EMMC/UFS: scp1/scp2
 - scp1: Main and active partitions
 - scp2: Backup for the AB system
- Image: scp.img
 - Consists of:
 1. tinysys-scp-RV55_A.bin: firmware/data located in SRAM
 2. tinysys-scp-RV55_A.elf: elf with symbol, for debug purpose
 3. tinysys-scp-RV55_A_DRAM.bin: firmware/data located in DRAM

5 Boot Sequence

After the SCP image is ready, it is a need to know how the image is loaded into SRAM/DRAM and how SCP is initiated to run.

The whole flow is completed by the LK bootloader/Linux kernel/SCP firmware together, as shown below:

- LK Bootloader
(`vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/mediatek/common/scp/scp.c`)
 - Allocate permanent DRAM memory for the SCP image
 - Load/verify SCP image
 - Set EMI MPU (AP read-only)
- Kernel
 - Initial setting (`mbox/ipi/logger/...`)
 - Kick SCP
- SCP
 - Loader
(`vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/boot55.s`)
 - Load the SCP image into SRAM
 - Jump to FreeRTOS
 - FreeRTOS
(`vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/src/main.c`)
 - Driver init
 - Set up MPU

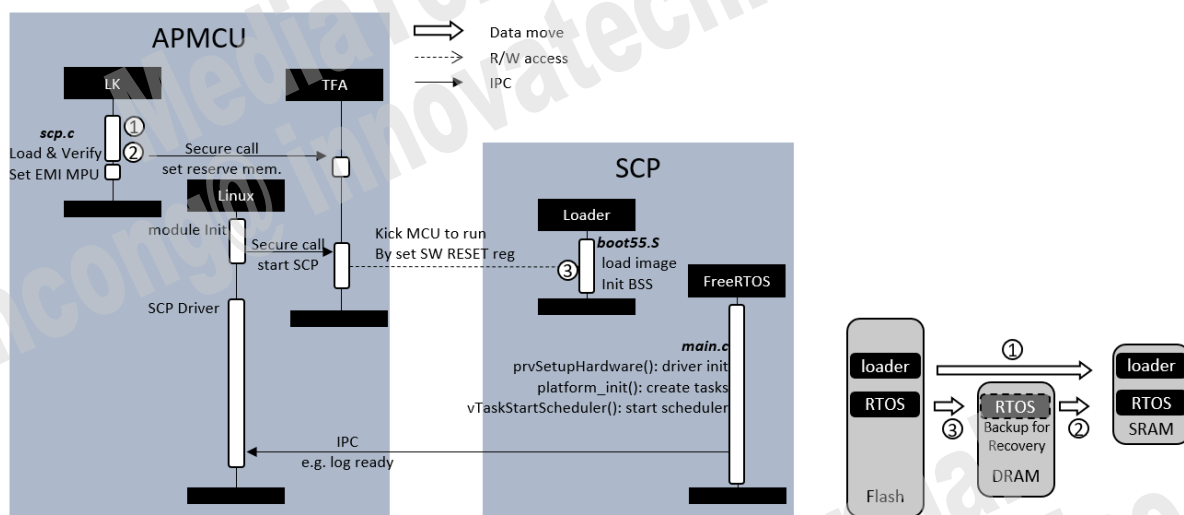


Figure 5-1. SCP boot up flow overview

6 Inter Processor Interrupt (IPI)

IPI is a mechanism to pass messages between drivers in Linux and FreeRTOS. It consists of:

1. A piece of shared memory to exchange data
2. A set of interrupts to notify each other

Figure 6-1 shows the Tinsys IPI architecture. The SW architecture consists of four layers

1. Synchronization layer: Public API for communication between AP and Tinsys
2. Rpmmsg layer: Blocking/non-blocking send functions
3. Queue layer: Queue operation and management functions
4. Physical layer: Physical hardware operations

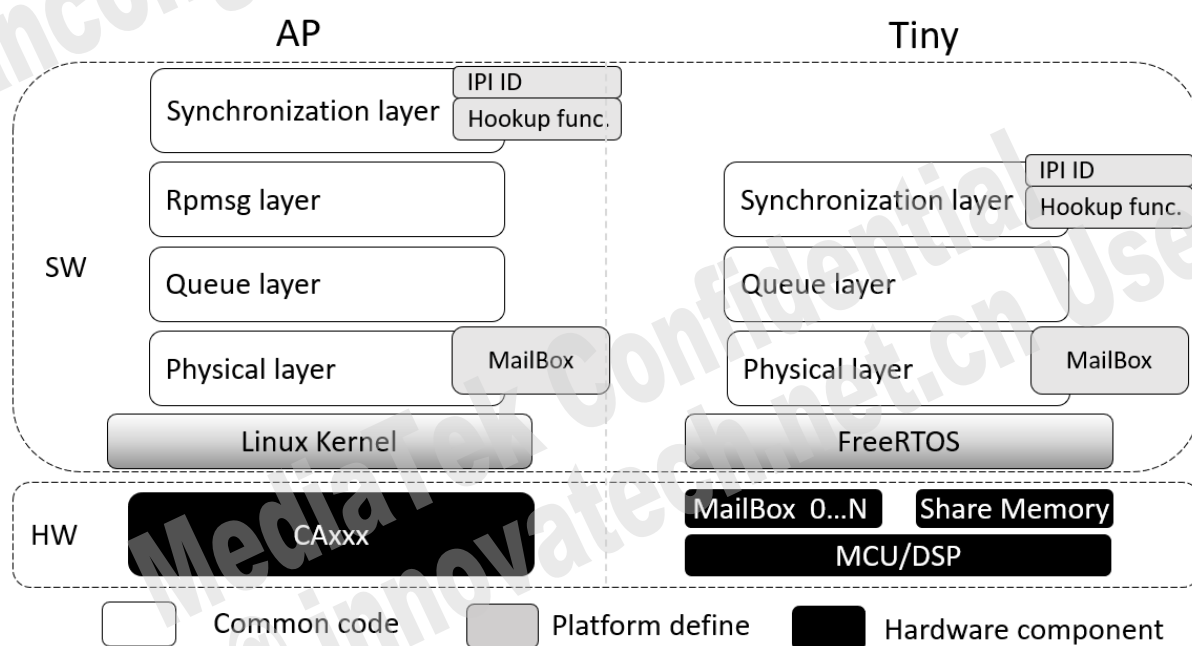


Figure 6-1. Tinsys common IPI architecture

The MailBox is only predefined. Developers use the flowing API to send IPI to Tinsys, including IPI ID and registering the call back function (ipi_cb).

6.1 Usage on the SCP Side

- Add new IPI ID information to `ipi_id.h`, `ipi_table.h`, and `mbox_pin.h`. Sending and receiving information are described by different IPI tables.
 - File Description**
 - `ipi_id.h`: The definition of the IPI ID (must be unique)
 - `ipi_table.h`: The definition of the IPI pin table (The sum of pin slots cannot exceed mbox slots.)
 - `mbox_ipi.h`: The definition of IPI pin slot
 - Path**
 - `vendor/mediatek/proprietary/tinsys/scp/drivers/RV55_A/mt8189/mbox/ipi_id.h`
 - `vendor/mediatek/proprietary/tinsys/scp/drivers/RV55_A/mt8189/mbox/ipi_table.h`
 - `vendor/mediatek/proprietary/tinsys/scp/drivers/RV55_A/mt8189/mbox/mbox_pin.h`

– IPI_TABLE (Sender Pin Table Description)

```

/* mbox pin structure, this is for send definition,
 * ipi=endpoint=pin
 * mbox:          (mbox number)mbox number of the pin, up to 16
 * offset:        (slot)msg offset in share memory, 4 bytes alignment, up to 1024*4 KB
 * send_opt:      (opt)send opt, 0:send ,1: send for response
 * lock_opt:      (opt)option 0: mutex, 1: busy wait
 * msg_size:      (slot)message size in words, 4 bytes alignment
 * pin_index:     (bit offset)pin index in the mbox
 * ipi_id:        (ipi_id)ipi_id in the mbox
 * mutex:         (mutex)mutex for remote response
 * sema_ack:      (sema_ack)completion for remote response
 * send_record:   send pin record information
 */
struct pin_send {
    unsigned int mbox:4,
        offset:20,
        send_opt:2,
        lock_opt:2;
    unsigned int msg_size;
    unsigned int pin_index;
    unsigned int ipi_id;
    SemaphoreHandle_t mutex;
    SemaphoreHandle_t sema_ack;
    struct mtk_mbox_send_record send_record;
};

```

– IPI_TABLE (Receiver Pin Table Description)

```

/*
 * mbox pin structure, this is for receive definition,
 * ipi=endpoint=pin
 * mbox:          (mbox number)mbox number of the pin, up to 16
 * offset:        (slot)msg offset in share memory, 4 bytes alignment, up to
1024*4 KB
 * recv_opt:      (opt)recv option, 0: receive ,1: response
 * lock_opt:      (opt)option 0: mutex, 1: busy wait
 * buf_full_opt:  (opt)buffer option 0:drop, 1:assert, 2:overwrite
 * cb_ctx_opt:    (opt)callback option 0:isr context, 1:process context
 * msg_size:      (slot)msg used slots in the mbox, 4 bytes alignment
 * pin_index:     (bit offset)pin index in the mbox
 * ipi_id:        (ipi_id)ipi_id in the mbox
 * notify:        (completion)notify process
 * mbox_pin_cb:   (cb)cb function
 * pin_buf:       (void*)buffer point
 * prdata:        (void*)private data
 * recv_record:   receive pin record information
 */
struct pin_recv {
    unsigned int mbox:4,
        offset:20,
        recv_opt:2,
        lock_opt:2,

```



```

        buf_full_opt:2,
        cb_ctx_opt:2;
    unsigned int msg_size;
    unsigned int pin_index;
    unsigned int ipi_id;
    SemaphoreHandle_t notify;
    void (*mbox_pin_cb) (unsigned int id, void *prdata, void *data, unsigned int len);
    void *pin_buf;
    void *prdata;
    struct mtk_mbox_recv_record recv_record;
};

```

Example of adding a new IPI ID

```

##Add IPI_NEW_ID at ipi_id.h
enum {
    .
    .
    IPI_NEW_ID =39,
    IPI_COUNT
};

```

Example of adding new IPI message size

```

##Add PIN_IPI_NEW_ID_SIZE at mbox_pin.h

#define PIN_IPI_NEW_ID_SIZE 2

```

Example of adding a new IPI pin

The following guidelines must not be violated.

- The total msg_size of the send_table and recv table in the same mbox must not exceed 64.
- The new ping setting must be bundled with the same mbox number

```

##Add new pin setting for send or recv table at ipi_table.h

Struct pin_send mbox_pin_send_table [] = {
    {0, 0, 0, 0, ..., {}},
    .
    .
    {0, 0, 0, 0, PIN_IPI_NEW_ID_SIZE, 0, IPI_NEW_ID, 0, 0, {}},
    {1, 0, 0, 0, ..., {}},
    .
    .
    {2, 0, 0, 0, ..., {}},
    .
    .
};

```

ii. Register an IPI handler

API	int ipi_register(unsigned int ipi_id, void *cb, void *prdata, void *msg)
Description	To register an IPI handler
Parameters	- id: ID declared in ipi_id.h - cb: IPI handler, a callback function - prdata: IPI handler parameter, for customization - msg: Message buffer, data received from AP
Return values	- DONE: Complete successfully - ERROR: Something is wrong in the lower layer driver, i.e. mbox - BUSY: The channel is busy. Need to retry.

Example

```
#include "scp_ipi.h"

int ret;
ret = ipi_register(IPI_NEW_ID, ipi_cb, 0, "ipi_cb name");
if (ret != IPI_ACTION_DONE)
    PRINTF_E("Register IPI failed\n");

void ipi_cb(int id, void *prdata, void *data)
{
    /* data: received message from kernel */
    unsigned int rcv_data = *(unsigned int *)data;
    ...
}
```

iii. Receive an IPI

API	int ipi_rcv(unsigned int ipi_id)
Description	API for APPs to receive an IP address from AP
Parameters	id: IPI ID declared in ipi_id.h
Return values	- DONE: Complete successfully - ERROR: There is something wrong in the lower layer driver, i.e. mbox.

Example:

```
#include "scp_ipi.h"

static void xxx_ipi_task(void *pvParameters)
{
    int ret;
    while(1) {
        ret = ipi_rcv(IPI_NEW_ID);
        If (ret != DONE)
            PRINTF_E("Send IPI failed\n");
    }
}
```

API	int ipi_recv_reply(unsigned int ipi_id, void *reply_data, int len)
Description	API for APPs to receive an IPI from AP and reply
Parameters	- id: IPI ID declared in ipi_id.h - reply_data: The message that will be replied to the Linux kernel - len: Message length (4 bytes as a unit)
Return values	- DONE: Complete successfully - ERROR: Something wrong in the lower layer driver, i.e. mbox

Example:

```
#include "scp_ipi.h"

static void new_ipi_task(void *pvParameters)
{
    int ret;
    while(1) {
        ret = ipi_recv_reply(IPI_NEW_ID, (void *)&reply_data, reply_data_len);
        If (ret != DONE)
            PRINTF_E("Send IPI failed\n");
    }
}
```

iv. Send an IPI

API	int ipi_send(unsigned int ipi_id, void *data, int len, unsigned long retry_timeout)
Description	API for APPs to send an IPI to the AP
Parameters	- id: IPI ID declared in ipi_id.h - data: The message which will be sent to the Linux kernel - len: Message length (4 bytes as a unit) - retry_timeout: The number of retries
Return values	- DONE: Complete successfully - ERROR: Something wrong in the lower layer driver, i.e., mbox

Example

```
#include "scp_ipi.h"

ipi_status ret;
ret = ipi_send (IPI_NEW_ID, (void *)&msg, msg_size, 0, IPI_SCP2AP);
if (ret != DONE)
    PRINTF_E("Send IPI failed\n");
```

v. IPI_STATUS

If any problem occurs in run-time usage, please check API return value first. For return value, refer to the following table.

#define IPI_ACTION_DONE	0	
#define IPI_ILLEGAL	-1	/* illegal ipi index */
#define IPI_DUPLEX	-2	/* the ipi has be registered */
#define IPI_UNAVAILABLE	-3	/* can't find this ipi pin define */
#define IPI_NO_MSGBUF	-4	/* ipi receiver doesn't has message buffer */
#define IPI_NO_MEMORY	-5	/* the message length is large than defined */
#define IPI_PIN_BUSY	-6	/* send message timeout */
#define IPI_RECV_TIMEOUT	-7	/* receive message timeout */
#define IPI_MBOX_ERR	-99	/* some error from physical layer */

6.2 Usage on the Kernel Side

i. Add a new IPI ID and pin table in scp.h and mt8189.dts

- File Description

- scp.h: The definition of IPI (ID must synchronize with the SCP side)
- mt8189.dts: The definition of the pin table (pin table must synchronize with the SCP side)

- Path

- kernel/kernel_device_modules-6.1/drivers/misc/mediatek/scp/include/scp.h
- kernel/kernel_device_modules-6.1/arch/arm64/boot/dts/mediatek/mt8189.dts

- SCP IPI Description in DTS

- Example of adding new IPI ID and message size

```
## Add new IPI ID and message size at scp.h
```

```
.
.
.
#define PIN_IPI_NEW_ID_SIZE      2
.
.
.
enum {
    .
    .
    IPI_NEW_ID = 39,
    SCP_IPI_COUNT
}
```

- Example of adding new IPI pin setting

```
scp: scp@1c400000 {
    .
    .
    send_table =
    // <id, mbox, send_size>
    .
    .
    <..>;
}
```

```

recv_table =
//< id, mbox, recv_size, recv_opt>,
< 39, 0, 2, 0>,
.
.
<..>;
}

```

ii. Register an IPI handler

API	int mtk_ipi_register(struct mtk_ipi_device *ipidev, int ipi_id, mbox_pin_cb_t cb, void *prdata, void *msg)
Description	To register an IPI handler
Parameters	- ipidev: ipidev declared in scp.h - ipi_id: ID declared in scp.h - cb: IPI handler, a callback - prdata: IPI handler parameter, for customization - msg: Message buffer, data received from SCP
Return values	- DONE: Complete successfully - ERROR: Something wrong in the lower layer driver, i.e. mbox

iii. Send an IPI

API	int mtk_ipi_send(struct mtk_ipi_device *ipidev, int ipi_id, int opt, void *data, int len, int timeout)
Description	API for APPs to send an IPI to SCP
Parameters	- ipidev: ipidev declared in scp.h - ipi_id: IPI id is declared in scp.h - opt: IPI_mode, IPI_SEND_WAIT, or IPI_SEND_POLLING, should be in sync with the pin table - data: The message which will be sent to SCP - len: Message size (4 bytes as a unit) - timeout: Busy wait time, as ms per unit
Return values	- DONE: Complete successfully - ERROR: Something wrong in the lower layer driver, i.e., mbox - SCP_NOT_READY: SCP is not ready. Need to retry.

iv. Receive an IPI

API	int mtk_ipi_rcv(struct mtk_ipi_device *ipidev, int ipi_id)
Description	API for APPs to receive an IPI from SCP
Parameters	- ipidev: ipidev declared in scp.h - ipi_id: IPI ID declared in scp.h
Return values	- DONE: Complete successfully - ERROR: Something is wrong in the lower layer driver, i.e. mbox - SCP_NOT_READY: SCP is not ready. Need to retry.

7 DRAM Access Procedure

Because DRAM and the system bus could be turned off regardless SCP's status, SCP developers **MUST** follow the following procedure to avoid system hang. Overview of sharing the information by DRAM between AP and SCP, shown as below.

(Notice: please refer to [Section 7.3](#) to get reserved DRAM address on the SCP side.)

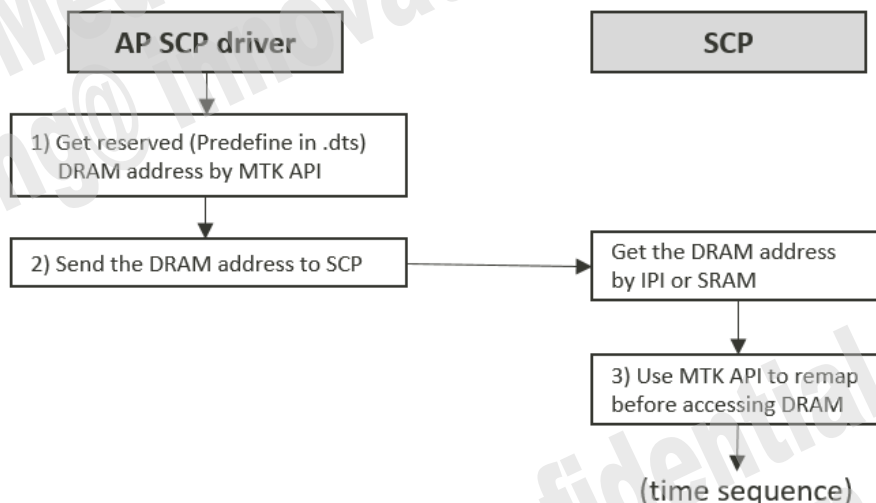


Figure 7-1. DRAM access procedure

Table 7-1. The standard DRAM access flow

#	Steps	Note
0	Check/add predefined table in <code>scp_reserve_mblock[]</code>	–
1	SCP Linux driver gets reserved physical address by APIs.	<code>phys_addr_t scp_get_reserve_mem_phys(scp_reserve_mem_id_t id)</code> <code>phys_addr_t scp_get_reserve_mem_virt(scp_reserve_mem_id_t id)</code> <code>phys_addr_t scp_get_reserve_mem_size(scp_reserve_mem_id_t id)</code>
2	The SCP Linux driver sends the address to the SCP via IPI.	<code>scp_ipi_send(uint32 id, void* buf, uint len)</code>
3	SCP accesses DRAM via an API.	<code>uint32_t ap_to_scp(uint32_t ap_addr)</code>
4	Enable DRAM before using it.	<code>void dvfs_enable_DRAM_resource(scp_reserve_mem_id_t dma_id)</code> <code>void dvfs_disable_DRAM_resource(scp_reserve_mem_id_t dma_id)</code>

7.1 Reserving Memory in Linux Kernel

To reserve a space in DRAM to exchange data between AP and SCP, it is a need to add entries in `scp_reserve_mblock[]` first. The definition could be found here:

- **Path**
 - `{lk2_repo}/platform/mediatek/mt8189/rules.mk`
 - `{kernel_repo}/arch/arm64/boot/dts/mediatek/mt8189.dts`
 - `{kernel_repo}/drivers/misc/mediatek/scp/include/scp.h`
 - `{kernel_repo}/drivers/misc/mediatek/scp/rv/scp_reservedmem_define.h`

7.1.1 Register an User to Shared Memory

step1. Add a new ID in scp_reserve_mem_id_t

```
{kernel repo}/drivers/misc/mediatek/scp/include/scp.h
enum scp_reserve_mem_id_t {
    SCP_A_SECDUMP_MEM_ID = 0,
    VOW_MEM_ID,
    SENS_MEM_ID,
    ...
    USER_MEM_ID, // add here
    NUMS_MEM_ID,
}
```

step2. Add a member to the data structure in scp_reserve_mblock

```
{kernel repo}/drivers/misc/mediatek/scp/rv/scp_reservedmem_define.h
static struct scp_reserve_mblock scp_reserve_mblock[] = {
    {
        num = SCP_A_SECDUMP_MEM_ID,
        start_phys = 0,
        start_virt = 0,
        size = 0,
    },
    {
        num = VOW_MEM_ID,
        start_phys = 0x0,
        start_virt = 0x0,
        size = 0x0,
    },
    ...
    {
        num = USER_MEM_ID,
        start_phys = 0x0,
        start_virt = 0x0,
        size = 0x0,
    },
}
```

step3. Add the dts node for reserved memory

```
{kernel repo}/arch/arm64/boot/dts/mediatek/mt8189.dts
scp: scp@1c400000{
    ...
    scp_mem_tbl = <0 0x0>,
                <1 0xca700>, /* vow */
                <2 0x100000>, /* sensor main*/
                ...,
                id, size>; /*user_mem*/
};
```


step4. Add memory size increment of SCP shared memory in the make file

```
{lk2_repo}/platform/mediatek/mt8189/rules.mk
SCP_RESERVED_SHARE_DRAM_SIZE:= 0x          //add increment to this variable
```

7.1.2 Extend the Shared Memory Size

Take Sensor as an example. Add 1.5MB.

```
{kernel_repo}/arch/arm64/boot/dts/mediatek/mt8189.dts
scp: scp@1c400000{
    ...
    scp_mem_tbl = <0 0x0>,
                <1 0xca700>, /* vow */
                <2 0x280000>, /* sensor main original value is 0x100000 */
    ...
};
```

```
{lk2_repo}/platform/mediatek/mt8189/rules.mk
SCP_RESERVED_SHARE_DRAM_SIZE:= 0x680000    // original = 0x500000
```

7.2 Get Reserved Memory by ID

The CPU of the AP uses a virtual address, while the SCP utilizes a physical address; both types of addresses are provided. For the specified APIs retrieve virtual or physical addresses and sizes, refer to [Section 7.1](#).

- **API**
 - phys_addr_t scp_get_reserve_mem_phys(scp_reserve_mem_id_t id)
 - phys_addr_t scp_get_reserve_mem_virt(scp_reserve_mem_id_t id)
 - phys_addr_t scp_get_reserve_mem_size(scp_reserve_mem_id_t id)
- **Header path**
 - kernel/kernel_device_modules-6.1/drivers/misc/mediatek/scp/rv/scp_helper.h
- **Return value**
 - The start address of the reserved memory, or
 - 0x0 means no mapping.

After getting the physical address, developers **MUST** pass it to SCP via IPI, as mentioned in [Section 6.1](#).

7.3 Get Reserved DRAM Region on SCP Side

To keep the security state and avoid malicious IPI messages, SCP_DRAM_REGION is introduced to provide the information of the reserved DRAM region on SCP directly. The following APIs are used to get the physical address and size by given IDs which are declared in [Section 7.1](#).

- **API**
 - bool scp_get_reserve_mem_by_id(uint32_t id, void **ap_addr, size_t *size)
- **Header path**

- vendor/mediatek/proprietary/tinysys/scp/drivers/common/dram_region_mgmt/scp_dram_region.h
- **Return value**
 - True: The value of *ap_addr and size are updated to start address and size of given ID.
 - False: The *ap_addr and size are not updated.

For the codes which may be common part on different projects, CFG_SCP_DRAM_REGION_MANAGE can help to keep compatibility. For example,

```
#ifndef CFG_SCP_DRAM_REGION_MANAGE
#include "scp_dram_region.h"
#define USER_MEM_ID 11 //id number from dts
#endif

void init_memory(....) {
...
#ifdef CFG_SCP_DRAM_REGION_MANAGE
    void *addr;
    size_t size;

    if (scp_get_reserve_mem_by_id(USER_MEM_ID, &addr, &size))
        set_dram_addr(addr, size);
    else
        ...
#else
    // msg.addr and msg.size from ipi message
    set_dram_addr(msg.addr, msg.size);
#endif
...
}
```

7.4 Remap DRAM Address from AP View to SCP View

SCP is a 32-bit system and can only access a maximum of 4G (0xffffffff). If SCP applications need to access more than 4G, it is a need to use the following APIs to remap the address to the SCP memory window. Here is a mapping table that shows the relation between AP and SCP.

Table 7-2. The default remapping rule table

Name	SCP/DMA		Size	AP Side	
External Memory	0x1000_0000	0x1FFF_FFFF	256MB	0x6000_0000	0x6FFF_FFFF
External Memory	0x2000_0000	0x2FFF_FFFF	256MB	0x7000_0000	0x7FFF_FFFF
External Memory	0x5000_0000	0x5FFF_FFFF	256MB	0x0000_0000	0x0FFF_FFFF
External Memory	0x6000_0000	0x6FFF_FFFF	256MB	0x1000_0000	0x1FFF_FFFF
External Memory	0x9000_0000	0x9FFF_FFFF	256MB	0x8000_0000	0x8FFF_FFFF
External Memory	0xA000_0000	0xAFFF_FFFF	256MB	0x9000_0000	0x9FFF_FFFF
External Memory	0xD000_0000	0xDFFF_FFFF	256MB	0x2000_0000	0x2FFF_FFFF
External Memory	0xE000_0000	0xEFFF_FFFF	256MB	0x3000_0000	0x3FFF_FFFF
External Memory	0xF000_0000	0xFFFF_FFFF	256MB	0x5000_0000	0x5FFF_FFFF

- **API**
 - `uint32_t ap_to_scp(uint32_t ap_addr);`
 - `uint32_t scp_to_ap(uint32_t scp_addr);`
- **Header path**
 - `vendor/mediatek/proprietary/tinysys/scp/drivers/common/dma/inc/dma_api.h`
- **Return value**
 - The mapped address, or
 - 0x0: means no mapping is done

7.5 Request System Bus and DRAM

Because the system bus and DRAM will enter sleep mode when no data is being transmitted or the system suspends, the following APIs **MUST** be invoked to make sure DRAM is ready to access.

- **API for task context**
 - `void dvfs_enable_DRAM_resource(scp_reserve_mem_id_t dma_id):` before DRAM access
 - `void dvfs_disable_DRAM_resource(scp_reserve_mem_id_t dma_id):` after DRAM access

Note: It will take 5ms to wake up DRAM when 26MHz clock is gated.

- **API for the ISR context**
 - `void dvfs_enable_DRAM_resource_from_isr(scp_reserve_mem_id_t dma_id)`
 - `void dvfs_disable_DRAM_resource_from_isr(scp_reserve_mem_id_t dma_id)`
- **Header path**
 - `vendor/mediatek/proprietary/tinysys/common/drivers/dma/v3/inc/dma.h`
 - `vendor/mediatek/proprietary/tinysys/scp/drivers/RV55_A/mt8189/dvfs/inc/dvfs.h`

8 Drivers Guide

8.1 Driver Initiation

Due to the fact that multi-threading is not enabled when the driver init, developers **MUST** do the following:

- Put drivers init function in `platform_init()`.
- NEVER use block functions in the driver init function because it will be blocked forever. For example,
 - `vTaskDelay`
 - HW semaphore
 - Busy loop, e.g., polling registers

8.2 Add a New Driver

Here are the steps to add a new driver to the source tree:

1. Put the driver body in the appropriate folder (choose one of the following folders)
 - Path
 - `vendor/mediatek/proprietary/tinysys/common` /* common drivers for tinysys */
 - `vendor/mediatek/proprietary/tinysys/scp/drivers/common` /* common drivers for scp */
 - `vendor/mediatek/proprietary/tinysys/scp/drivers/RV55_A/mt8189/drivers` /* platform drivers */
2. Add a new compiler option
 - Path
 - `vendor/mediatek/proprietary/tinysys/scp/project/mt8189/platform/platform.mk`

Example: DMA driver

```
CFG_DMA_SUPPORT = yes
...
ifeq ($(CFG_DMA_SUPPORT), yes)
INCLUDES += $(COMMON_DIR)/drivers/dma/v3/inc
INCLUDES += $(SCP_DRIVERS_DIR)/common/dma/inc
INCLUDES += $(DRIVERS_PLATFORM_DIR)/dma
C_FILES += $(COMMON_DIR)/drivers/dma/v3/dma.c
C_FILES += $(SCP_DRIVERS_DIR)/common/dma/dma_api.c
endif
```

8.3 Interrupt

MT8391 SCP supports 15 priority levels of interrupt, and the lower-level number has the higher priority. If more than one IRQ happens at the same time, the CPU will serve the one with the highest priority first.

NEVER set interrupt priorities higher than 2. Level 0 is for the “watch dog” or “system fail”, level 1 is for “sleep control” interrupt.

This section will describe how to use interrupts in SCP.

- **Path**
vendor/mediatek/proprietary/tinysys/common/drivers/irq/v3/inc/irq.h

8.3.1 IRQ Registration

Before an IRQ is served, the driver developer must make an association between the IRQ ID and the corresponding handler. The IRQ is defined according to the structure of INTC_IRQ, and the driver developer follows the API to register handlers with IRQ IDs.

- **Structure**

```
struct INTC_IRQ
{
    uint8_t id;
    uint8_t group;
    uint8_t pol;
}
```

Where,

- id: The IRQ number
- group: From INTC_GRP_0 (the highest priority) to INTC_GRP_14 (the lowest priority)
- pol: The polarity is INTC_POL_HIGH or INTC_POL_LOW

- **API**

API	int intc_irq_request(struct INTC_IRQ *irq, irq_handler_t handler, void *userdata)
Description	Request an IRQ and register the handler, and enable the IRQ handler by default.
Parameters	• irq: The IRQ structure that is declared in intc.h • handler: IRQ handler
Return values	• 0: Success • -1: Request failed

- **Example**

```
struct INTC_IRQ INTC_IRQ_SYSTICK = {0, INTC_GRP_8, INTC_POL_HIGH};

#include "irq.h"

int ret;
ret = intc_irq_request(&INTC_IRQ_SYSTICK, test_ist, NULL);
if (ret != 0)
    PRINTF_E("Register irq failed\n");
```

8.3.2 Enable IRQ

After the IRQ handler is registered, SCP is ready to serve. The next step is to enable a specific IRQ with the following API.

- API

API	int intc_irq_enable(struct INTC_IRQ *irq)
Description	Enable specified IRQ served.
Parameters	irq: The IRQ structure declared in intc.h.
Return values	0: Success -1: Request failed

- Example:

```
#include "irq.h"

int ret;
ret = intc_irq_enable(&INTC_IRQ_SYSTICK);
if (ret != 0)
    PRINTF_E("enable irq failed\n");
```

8.3.3 Disable IRQ

On the contrary, when IRQs are not used temporarily, driver developers have to call the following API to make SCP stop servicing the IRQs.

- API

API	int intc_irq_disable(struct INTC_IRQ *irq)
Description	Disable the specified IRQ.
Parameters	irq: The IRQ structure, which is declared at intc.h
Return values	0: Success -1: Request failed

- Example

```
#include "irq.h"

int ret;
ret = intc_irq_disable(&INTC_IRQ_SYSTICK);
if (ret != 0)
    PRINTF_E("disable irq failed\n");
```

8.3.4 Wakeup Source Registration

When SCP is in the sleep state, an IRQ will not be serviced unless it is set as a wakeup source. The following API is used to register an IRQ as a wakeup source.

• API

API	int intc_irq_wakeup_set(struct INTC_IRQ *irq, unsigned int wake_src)
Description	Set wakeup source for specified IRQ
Parameters	- irq: The IRQ structure which is declared in intc.h - wake_src: 1 for wakeup source, 0 for non-wakeup source
Return values	0: Success -1: Request failed

• Example

```
#include "irq.h"

int ret;
ret = intc_irq_wakeup_set(&INTC_IRQ_SYSTICK, 1);
if (ret != 0)
    PRINTF_E("irq wakeup source setup failed\n");
```

Note:

- ISR execution time must be as **SHORT** as possible and stack depth as **LESS** as possible.
- **NEVER** use block APIs, such as
 - HW semaphore API
 - Wait for IPI
 - Any APIs with polling external devices
- **NEVER** use FreeRTOS API in ISR without FromISR postfix
 - It's a rewritten version with block-free API and quick implementation
- **MUST** use portYIELD_FROM_ISR() if there is a high priority task waken in ISR

8.4 Locks

SCP provides spin_lock mechanism for dual core synchronization. To use the lock, the following configuration item needs to be enabled.

```
Set CFG_ATOMIC_PLAT_SUPPORT = yes
```

• Path

```
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/platform.mk
```

• API

API	<pre>spinlock_t SYNC_SECTION lock; void spinlock_lock(spinlock_t * lock) void spinlock_unlock(spinlock_t * lock)</pre>
-----	---

Description	Spin lock for synchronous dual core
Parameters	lock: A variable defined by SYNC_SECTION
Return values	NA

- Header path**

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/inc/mtk_atomic.h

Note:

- NEVER** keep spin lock more than 1ms because preempt is disabled during spin lock and unlock.
- For those scenarios which do not cross dual cores, spin locks are not necessary.

8.5 DMA

Direct Memory Access (DMA) is a kind of hardware that supports copy data from/to specified source/destination without involving CPU.

SCP DMA supports:

- Burst AXI mode to speed up memory translations
- Eight channels, i.e., the engine could process up to 8 transactions simultaneously.

- API**

API	DMA_RESULT scp_dma_transaction(uint32_t dst_addr, uint32_t src_addr, uint32_t len, int8_t scp_dma_id, int32_t ch) DMA_RESULT scp_dma_transaction_dram(uint32_t dst_addr, uint32_t src_addr, uint32_t len, int8_t scp_dma_id, int32_t ch)
Description	Copy data from the src_addr to the dst_addr using the specified DMA channel
Parameters	- dst_addr: Destination address - src_addr: Source address - len: Length in bytes to copy - scp_dma_id: DMA ID - ch: Channel ID
Return values	- DMA_RESULT_DONE (=0) means success start - DMA_RESULT_NO_FREE_CH (=1) means DMA hardware busy

- Header path**

- vendor/mediatek/proprietary/tinysys/common/drivers/dma/v3/inc/dma.h
- vendor/mediatek/proprietary/tinysys/scp/drivers/common/dma/dma_api.h
- vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/mt_dma.h

- scp_dma_id**

- It's the identity of DMA channel in mt_dma.h. It's recommended to use different dma_id with which debugging channel full issues will be easier.

- Limitations**

- The maximum data size per transaction is 262,140 bytes
- Use a 4-byte-aligned address to achieve the best performance

• **Example:**

```
ret = scp_dma_transaction(dst_buf + dst_w_pos, src_buf + src_r_pos, src_len, LOGGER_DMA_ID, NO_RESERVED);
if (ret != DMA_RESULT_DONE) {
    PRINTF_E("log dma trans fail%u\n", ret);
    return 0;
}
```

8.6 Hardware Semaphore

A hardware semaphore is specialized hardware that offers mutex-like flow control between a Linux driver and FreeRTOS. There are 16 sets in SCP. The following APIs make hardware semaphore easy to use. They work on both SCP and Linux drivers. Just include the right header and make sure the flags are the same.

• **API**

API	int semaphore_get(unsigned int flags); int semaphore_release(unsigned int flags);
Description	Semaphore between AP and SCP
Parameters	• flag: 0 ~15 for 16 sets in SCP
Return values	• 0: Getting semaphore failed. • 1: Getting semaphore succeeded.

• **Header**

- kernel/kernel_device_modules-6.1/drivers/misc/mediatek/scp/rv/scp_helper.h
- vendor/mediatek/proprietary/tinysys/common/drivers/sem/v1/inc/sem.h

• **Return**

- 1: Successfully acquired semaphore
- 0: Failed to get semaphore

• **Example:**

```
int get_semaphore;

while(1) {
    get_semaphore = semaphore_get(4)
    if (get_semaphore)
        break;
}
```

8.7 GPIO & EINT

SCP also provides GPIO and external interrupts so that external components such as gyro sensors can send events to SCP.

- **GPIOs**
 - Function set to: TP_GPIO?_AO, support TP_GPIO0_AO~TP_GPIO7_AO
- **EINTs**
 - Function set to Aux Func.0(GPIO), support EINT0~17

Table 8-1. The EINT and GPIO mapping ball name

Ball Name	SCP_EINT	EINT	Aux Func.0	Aux Func.1
GPIO00	SCP_EINT0	EINT0	B:GPIO0	B0:TP_GPIO0_AO
GPIO01	SCP_EINT1	EINT1	B:GPIO1	B0:TP_GPIO1_AO
GPIO02	SCP_EINT2	EINT2	B:GPIO2	B0:TP_GPIO2_AO
GPIO03	SCP_EINT3	EINT3	B:GPIO3	B0:TP_GPIO3_AO
GPIO04	SCP_EINT4	EINT4	B:GPIO4	B0:TP_GPIO4_AO
GPIO05	SCP_EINT5	EINT5	B:GPIO5	B0:TP_GPIO5_AO
GPIO06	SCP_EINT6	EINT6	B:GPIO6	B0:TP_GPIO6_AO
GPIO07	SCP_EINT7	EINT7	B:GPIO7	B0:TP_GPIO7_AO
GPIO08	SCP_EINT8	EINT8	B:GPIO8	–
GPIO09	SCP_EINT9	EINT9	B:GPIO9	–
GPIO10	SCP_EINT10	EINT10	B:GPIO10	–
GPIO11	SCP_EINT11	EINT11	B:GPIO11	–
GPIO12	SCP_EINT12	EINT12	B:GPIO12	–
GPIO13	SCP_EINT13	EINT13	B:GPIO13	–
GPIO14	SCP_EINT14	EINT14	B:GPIO14	–
GPIO15	SCP_EINT15	EINT15	B:GPIO15	–
GPIO16	SCP_EINT16	EINT16	B:GPIO16	–
GPIO17	SCP_EINT17	EINT17	B:GPIO17	–

8.7.1 GPIO Usage

The GPIO function must be set to TP_GPIO?_AO first. Please refer to GPIO pin mux setting document for the detail. The GPIO control register table is as shown in [Table 8-2](#).

Table 8-2. The GPIO control register table

Register Offset	Name	Description	Access	Enumeration
0x25000	GPIO_DIR	GPIO direction	RW	0: Input 1: Output
0x25004	GPIO_OUT	GPIO output	RW	GPIO output[1:0]
0x25008	GPIO_IN	GPIO input	RO	GPIO input[1:0]
0x2500C	GPIO_PULL_EN	GPIO pull enable	RW	0: Disable it 1: Enable it

Register Offset	Name	Description	Access	Enumeration
0x25010	GPIO_PULL_CTRL	GPIO pull control	RW	0: Pull down 1: Pull up

The control examples:

- Pull high GPIO 1
 - Set GPIO_DIR, 0x60525000[1] = 1
 - Set GPIO_OUT, 0x65025004[1] = 1
- Read GPIO 0
 - Set GPIO_DIR, 0x60525000[0] = 0
 - Read GPIO_IN, 0x60525008[0]
- Internal pull up GPIO 3
 - Set GPIO_PULL_EN, 0x6052500c[3] = 1
 - Set GPIO_PULL_CTRL, 0x60525010[3] = 1

8.7.2 EINT Usage

- **Path**
vendor/mediatek/proprietary/tinysys/common/drivers/eint/v02/src
- **API:** Register EINT callback

API	void mt_eint_registration(unsigned int eint_num, unsigned int sens, unsigned int pol, void (EINT_FUNC_PTR) (int), unsigned int unmask, unsigned int is_auto_unmask)
Description	Register an interrupt handler
Parameters	• eint num: The EINT number to register • sens: LEVEL_SENSITIVE, EDGE_SENSITIVE • pol: HIGH_LEVEL_TRIGGER, LOW_LEVEL_TRIGGER • EINT_FUNC_PTR: The ISR callback function • Unmask: Enable this EINT trigger after you register • Is_auto_unmask: Automatically re-enable EINT trigger after finishing an EINT service routine
Return values	NA

- **Example**

```
mt_eint_registration(eint_num, LEVEL_SENSITIVE, HIGH_LEVEL_TRIGGER, xxx_Isr,
                    EINT_INT_UNMASK, EINT_INT_AUTO_UNMASK_OFF);
```

- void mt_eint_dis_hw_debounce(unsigned int eint_num): Disable HW debounce
- void mt_eint_soft_set(unsigned int eint_num): Software trigger to clear specified EINT

9 Debug Methods

9.1 PRINTF_* Usage

Please **DO NOT** use `printf` in SCP SW, because it could be linked to `C` library and cause problems. Instead, use `PRINTF_*` as Table 9-1 shows. Developers **MUST** include the header file `<mt_printf.h>` before using `PRINTF_*`. The log level and the use scenario of `PRINTF_x` shown below.

Table 9-1. PRINTF use scenario

PRINTF_*	Level	Use Scenario	Load
PRINTF_E	<0>	Error conditions	–
PRINTF_W	<1>	Warning conditions	User load
PRINTF_I	<2>	Informational	–
PRINTF_D	<3>	Debug-level messages	Engineer load

9.2 Mobile Log

MTK Logger is an APK that records various logs into storage such as SD card. After launching it and enabling SCP log, the SCP log can be found in the following path:

- Log path: `/mobilelog/APLog_XXXX_XXXX_XXXXXX/scp_log_XXXX.curf`

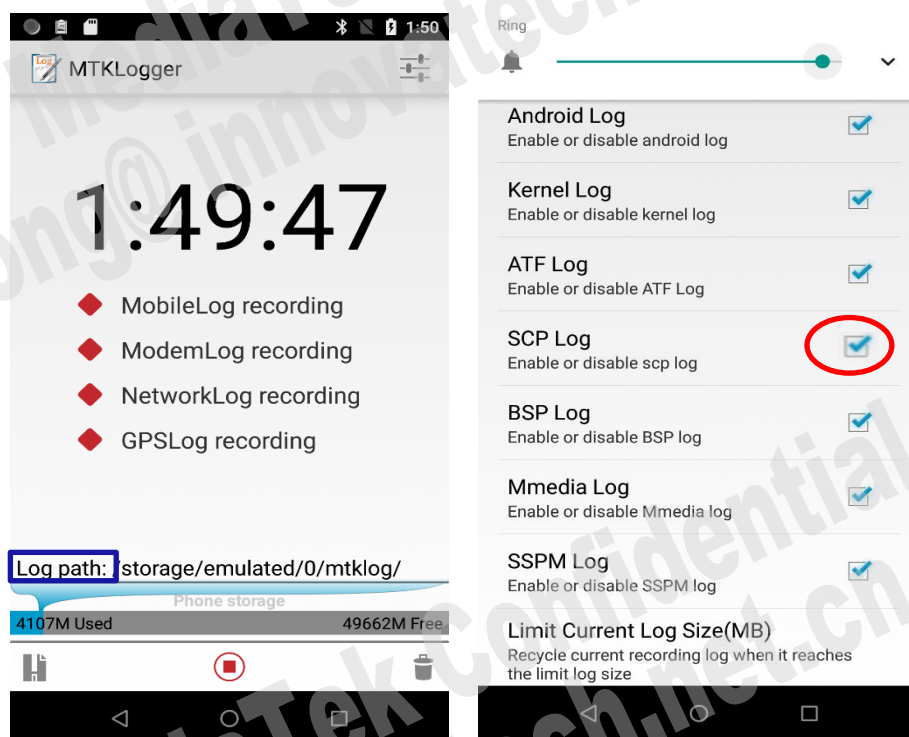


Figure 9-1. MTK Logger

9.3 UART

• Output pin

SCP has two dedicated UARTs. Please make sure the PC UART port is connected to pin ball name URTD1 and UTXD1 (as shown in Table 9-2) and then set the software compiler option as below.

Table 9-2. UART pin name

Pin Name	Function
URXD1	SCP UART RX
UTXD1	SCP UART TX

• SW compiler option

- Path: Configure flags
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/platform.mk
 - CFG_UART_SUPPORT = yes /* Uart enable, Default No */
 - CFG_MTK_SCPUART_SUPPORT = yes /* Use SCP uart, Default Yes*/
- UART terminal setting
 - Baud rate: 921600

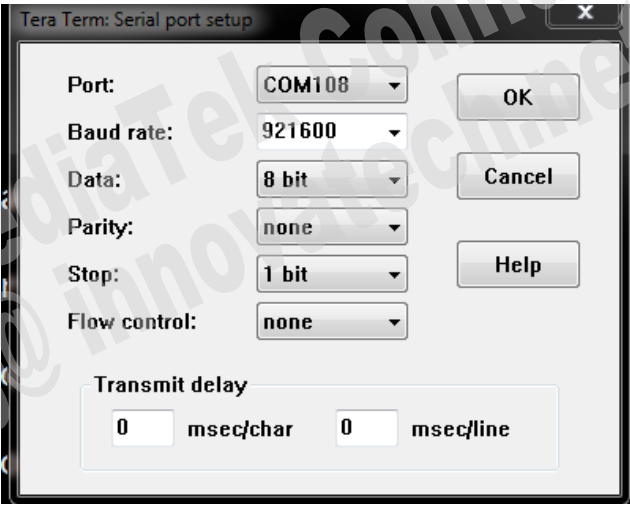


Figure 9-2. UART setting

9.4 ADB Logcat

ADB logcat is able to output an SCP log directly from the ADB or UART console.

- Usage:
 1. Make sure the SCP log in MTK Logger is disabled (as shown in Figure 9-3.)
 2. Enter shell and enter command “echo 1 > /sys/class/misc/scp/scp_mobile_log”
 3. Enter command: “while true; do cat /dev/scp;done” and the log output directly, as shown in Figure 9-4.

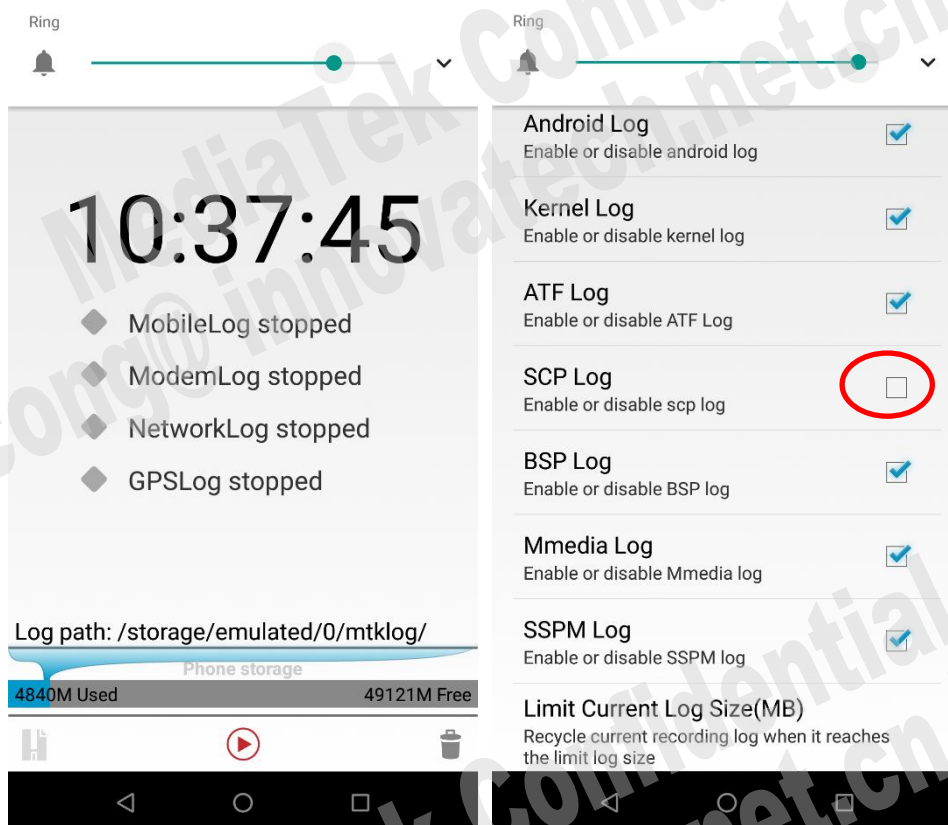


Figure 9-3. Disable SCP Mobile log

```
C:\Users\MTK11261>adb shell
k71v1 64 bsp: # echo 1 > /sys/class/misc/scp/scp_mobile_log
k71v1 64 bsp: # while true; do cat /dev/scp;done
103284, ap:39031836157017, ap_raw:39031836071402
raw_offset:2290053733, timestamp_offset_to_ap:2290053733
sync time counter elapse:1126, ipi_transfer_time:86615
sync time scp:39039562168462, ap:39041852222325, ap_raw:39041852135710
raw_offset:2290053863, timestamp_offset_to_ap:2290053863
No sleep reasons: tnr=0, build=0, sema=0, lock=0, ipi=0, flag=4, slpbusy=0
sync time counter elapse:1129, ipi_transfer_time:86846
sync time scp:39049578174332, ap:39051868228172, ap_raw:39051868141326
raw_offset:2290053840, timestamp_offset_to_ap:2290053840
sync time counter elapse:1130, ipi_transfer_time:86923
sync time scp:39059594165433, ap:39061884219173, ap_raw:39061884132250
raw_offset:2290053740, timestamp_offset_to_ap:2290053740
sync time counter elapse:1111, ipi_transfer_time:85461
sync time scp:39069610088456, ap:39071900142173, ap_raw:39071900056712
raw_offset:2290053717, timestamp_offset_to_ap:2290053717
No sleep reasons: tnr=0, build=0, sema=0, lock=0, ipi=0, flag=3, slpbusy=0
log en=1,update=1
sync time counter elapse:1208, ipi_transfer_time:92923
sync time scp:39079626695403, ap:39081916749174, ap_raw:39081916656251
raw_offset:2290053771, timestamp_offset_to_ap:2290053771
^C
```

Figure 9-4. ADB logcat output

9.5 Exception Log Analysis

When an exception occurs, the exception log will be printed automatically by SCP. It can be obtained by developers from either UART or the mobile log.

- FAULT FETCH, FAULT LOAD, and FAULT STORE: Due to the wrong access of a MPU protected address.
 - FAULT FETCH: The PC jumps to the wrong address.
 - The system register: mepc will show the incorrect target address.

– **FAULT LOAD, FAULT STORE:** Access a protected address

- The system register mepc will show the wrong PC, and mtval will show the incorrect address to be accessed.

```
[70.168] (0) exception: CAUSE_FAULT_LOAD
[70.168] (0) exception pc: 0x00018b2c
[70.168] (0) fault load address: 0xffff1110
[70.168] (0) Regs dump
[70.168] (0) x0: 0xca0801e4 ra: 0x00018b24
[70.168] (0) sp: 0x00036ec0 gp: 0x00000000
...
[70.168] (0) t3: 0x00000000 t4: 0x00000011
[70.168] (0) t5: 0x00000000 t6: 0x00000000
[70.168] (0) pc: 0x00001454 mstatus: 0x05015800
[70.168] (0) mepc: 0x00018b2c
[70.168] (0) mcause: 0x00000005
[70.168] (0) mtval: 0xffff1110
[70.168] (0) T Buffer (10)
[70.168] (0) 00 0x000fdca:0x0002252c
[70.168] (0) 01 0x0002252c:0x000fdce
...
[70.169] (0) 28 0x000108c0:0x0002368e
[70.169] (0) 29 0x00023698:0x000108c4
[70.169] (0) 30 0x000108ca:0x00024518
[70.169] (0) 31 0x00024524:0x00018b24
[70.169] (0) Code: 7545 85aa 0513 1105 <4108> 8593 1145 c188 9205
```

- **MISALIGNED FETCH, MISALIGNED LOAD, and MISALIGNED STORE:** Usually happen when the integer pointer (4 byte aligned) accesses a 1 or 2 bytes aligned address.

– **MISALIGNED FETCH:** Jump to a misaligned address

- The system register: mepc will show the wrong address.

– **MISALIGNED LOAD, MISALIGNED STORE:** Access a misaligned address

- The system register mepc will show the wrong PC, and mtval will show the access address.

- **ILLEGAL_INSTRUCTION:** Due to the instruction cannot be decoded

- For example:

```
[2.643] (0) exception: CAUSE_ILLEGAL_INSTRUCTION
[2.643] (0) Regs dump
[2.644] (0) x0: 0xa5a5a5a5 ra: 0xa5a5a5a5
[2.644] (0) sp: 0xa5a5a5a5 gp: 0xa5a5a5a5
...
[2.651] (0) t3: 0x00000000 t4: 0x00000000
[2.652] (0) t5: 0x00000000 t6: 0x00000000
[2.652] (0) pc: 0x00001454 mstatus: 0x05015880
[2.653] (0) mepc: 0x00201720
[2.654] (0) mcause: 0x00000005
[2.654] (0) mtval: 0x2ff13748
[2.655] (0) T Buffer (10)
[2.656] (0) 00 0x0000371a:0x00008fd6
[2.656] (0) 01 0x00008fdc:0x0000371e
...
[2.661] (0) 28 0x0000f2a4:0x00002eec
```

```
[2.662] (0) 29 0x00002eec:0x00006d52
[2.662] (0) 30 0x00006d5c:0x00002ef0
[2.663] (0) 31 0x0000f2a4:0x00003728
[2.664] (0) Code: d194 5133 4521 873f <3748> 2ff1 a5a5 a5a5 a5a5
```

9.5.1 Trace Buffer

Trace buffer log can help analyze the function call record that CPU has been executed. The log indicates the function call address and the destination address. There are 32 entries and can be gotten from the bottom of the exception log.

For example:

The address can be translated with tools like lldb or addr2line.

```
[2.655] (0) T Buffer (10)
[2.656] (0) 00 0x0000371a:0x00008fd6
[2.656] (0) 01 0x00008fdc:0x0000371e
...
[2.661] (0) 28 0x0000f2a4:0x00002eec
[2.662] (0) 29 0x00002eec:0x00006d52
[2.662] (0) 30 0x00006d5c:0x00002ef0
[2.663] (0) 31 0x0000f2a4:0x00003728
```

After executing the SCP_COREDUMP file via `coredump_cmd.sh`, a readable text file would be generated, `analysis.txt`.

The file list is a combo of function call address, function name, and file path of the source file.

```
C0 0: 0x0000371a: scp_ipi_queue_init at alps-dev-r0_mp3/vendor/.../.../adsp_ipi_queue.c:522
C0 1: 0x00008fd6: xQueueSemaphoreTake at alps-dev-r0_mp3/vendor/.../.../queue.c:1599
(discriminator 6)
C0 2: 0x00008fdc: scp_ipi_queue_init at alps-dev-r0_mp3/vendor/.../.../adsp_ipi_queue.c:500
C0 2: 0x0000371e: scp_sleep_ctrl at alps-dev-r0_mp3/vendor/.../.../sleep.c:377
```

9.6 Core Dump

If AP is aware that SCP is not responding to IPI or an SCP WDT event, the Core Dump flow will start automatically. Core Dump is a snapshot of the SCP memory, and the processor registers like program counter, stack pointer, and return address will be saved as well. And much of the system information like system registers, cache contents will be saved as much as possible. This information makes it possible to restore the status of the system before the fault happened.

By default, LLDB is offered to support core dump debug. For LLDB detail, please reference <https://lldb.lldm.org/>.

- **LLDB path**
LLDB could be found in `alps_prebuilts` folder, path "`prebuilts/clang/md32rv/linux-x86/lldb_v3`".
- **Get core dump**
The SCP core dump will be named as "`SYS_SCP_DUMP`", this will be inside SCP EE DB (e.g., `db.00.EE.dbg`).
If not found, please check the EE DB, confirm the `__exp_main.txt`, the exception type should be "`scp`".
- **Start debug**
- **Type the command below, and the initial log is shown below the command.**

```
$ prebuilts/clang/md32rv/linux-x86/lldb_v3/coredump_cmd.sh mt8189 tinysys-scp-RV55_A.elf
SCP_COREDUMP 0
```

```
$ addr2line -e tinysys-scp-RV55_A.elf -af 0xf2a4
turn_off_clk_sys_from_isr
alps/vendor/mediatek/proprietary/tinysys/scp/drivers/RV55_A/mt8189/dvfs/src/dvfs.c:449
```

```
(lldb) command script import
/proj/mtk11261/misc/coredump/scp_core_dump_cmdline/bin/coredump/freertos.py
(lldb) freertos

FreeRTOS Awareness is working.

CPU 0 -- OS Tick: 70125
CPU 1 -- OS Tick: 0
OS is in a normal state (neither in the Critical Section nor in HW ISR)
TASK-0: IDLE@cpu0 ( TCB: 0x00042c60, State: Running, TCB Number: 8, Priority: 0)
Stack (unit in words) size = 1024, currently used = 124, maximum used = 197
* frame #0: 0x000231c6 tinysys-scp-RV55_A.elf`mrv_core_dump(epc=-61168, regs=0xffff1000)
at exception.c:1055
    frame #1: 0x00003eb0 tinysys-scp-RV55_A.elf`atomicCmpXchg32bits(word=<unavailable>,
prevVal=238376, newVal=<unavailable>) at atomic.c:181
...
The Flight Recorder is working ...
No Flight Recorder Support
(lldb)
```

Note:

- LLDB version must be compatible with SCP core.
- RV55: lldb_v3
- RV33: lldb

If the LLDB tool was launched unexpectedly,

1. Please confirm the file structure under `prebuilts/clang/md32rv/linux-x86/lldb_v3` was not modified.
2. Please check the related log file “`debug_prosim.log`” and “`debug_ocd.log`”. If the `libprofile.so.x.x.x` libraries were not found, add the following command to “`coredump_cmd.sh`” before PROSIM launches command.

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$MRV_PDK_PROSIM_HOME

#add above this command
$PROSIM_EXE $PROSIM_OPT
```

3. There are three tools (LLDB, Openocd, PROSIM) launched when using “`coredump_cmd.sh`” and tools communicate each other via TCP/IP protocol. If remotely working with LLDB related tools, please confirm that the TCP/IP connect ports are not blocked by firewall.

9.6.1 LLDB Basic Commands

For the advanced developers or developers who are familiar with GDB, please refer to [command map](#) for the detail.

- Back trace: `bt`

```
(lldb) bt
* thread #1, name = 'Hart0', stop reason = instruction step into
```

```
* frame #0: 0x000231c6 tinysys-scp-RV55_A.elf`mrv_coredump(epc=-61168, regs=0xffff1000) at
exception.c:1055
  frame #1: 0x00003eb0 tinysys-scp-RV55_A.elf`atomicCmpXchg32bits(word=<unavailable>,
prevVal=238376, newVal=<unavailable>) at atomic.c:181
  frame #2: 0x0000befa tinysys-scp-RV55_A.elf`osSetCurrentTid [inlined]
osSetCurrentTask(task=0x0003a328) at seos.c:153
  frame #3: 0x0000bee6 tinysys-scp-RV55_A.elf`osSetCurrentTid(tid=<unavailable>) at
seos.c:212
  frame #4: 0x0000d872 tinysys-scp-RV55_A.elf`timFireAsNeededAndUpdateAlarms at
timer.c:145
  frame #5: 0x0000d608 tinysys-scp-RV55_A.elf`timTimerSetEx(length=<unavailable>,
jitterPpm=<unavailable>, driftPpm=<unavailable>, info=<unavailable>, data=<unavailable>,
oneShot=<unavailable>) at timer.c:179
  frame #6: 0x0000cf4e tinysys-scp-RV55_A.elf`osDefer(callback=<unavailable>,
cookie=<unavailable>, urgent=<unavailable>) at seos.c:1184
  frame #7: 0x000467f4 tinysys-scp-RV55_A.elf`ucHeap + 43796
```

• Dump OS tasks information: freertos

```
(lldb) freertos
```

```
Find 18 tasks
```

```
CPU 0 -- OS Tick: 70125
```

```
CPU 1 -- OS Tick: 0
```

```
Coredump cpu: 0
```

```
OS is in a normal state (neither in the Critical Section nor in HW ISR)
```

```
TASK-0: IDLE@cpu0 ( TCB: 0x00042c60, State: Running, TCB Number: 8, Priority: 0)
```

```
Stack (unit in word) size=1024, current used=124, max used=197
```

```
* frame #0: 0x000231c6 tinysys-scp-RV55_A.elf`mrv_coredump(epc=-61168, regs=0xffff1000)
at exception.c:1055
```

```
  frame #1: 0x00003eb0 tinysys-scp-RV55_A.elf`atomicCmpXchg32bits(word=<unavailable>,
prevVal=238376, newVal=<unavailable>) at atomic.c:181
```

```
  frame #2: 0x0000befa tinysys-scp-RV55_A.elf`osSetCurrentTid [inlined]
osSetCurrentTask(task=0x0003a328) at seos.c:153
```

```
  frame #3: 0x0000bee6 tinysys-scp-RV55_A.elf`osSetCurrentTid(tid=<unavailable>) at
seos.c:212
```

```
  frame #4: 0x0000d872 tinysys-scp-RV55_A.elf`timFireAsNeededAndUpdateAlarms at
timer.c:145
```

```
  frame #5: 0x0000d608 tinysys-scp-RV55_A.elf`timTimerSetEx(length=<unavailable>,
jitterPpm=<unavailable>, driftPpm=<unavailable>, info=<unavailable>, data=<unavailable>,
oneShot=<unavailable>) at timer.c:179
```

```
  frame #6: 0x0000cf4e tinysys-scp-RV55_A.elf`osDefer(callback=<unavailable>,
cookie=<unavailable>, urgent=<unavailable>) at seos.c:1184
```

```
  frame #7: 0x000467f4 tinysys-scp-RV55_A.elf`ucHeap + 43796
```

```
...
```

```
TASK-17: CHRE@cpu0 ( TCB: 0x0003f910, State: Suspended, TCB Number: 4, Priority: 4)
```

```
Stack (unit in word) size=1024, current used=132, max used=364
```

```
* frame #0: 0x00013bc4 tinysys-scp-RV55_A.elf`vTaskExitCritical at tasks_smp.c:4287
```

```
  frame #1: 0x00007aee tinysys-scp-RV55_A.elf`evtQueueDequeue(q=<unavailable>,
evtTypeP=<unavailable>, evtDataP=<unavailable>, evtFreeDataP=<unavailable>,
sleepIfNone=<unavailable>) at eventQ.c:171
```

```
frame #2: 0x0000cbb8 tinysys-scp-RV55_A.elf`osMainDequeueLoop at seos.c:1052
frame #3: 0x0000cdfc tinysys-scp-RV55_A.elf`osMain at seos.c:1093
```

- Print variable and address: p \$ (variable name)

```
(lldb) p mTask
(alsPsTask) $0 = {
    id = 259
    handle = ([0] = 16973827, [1] = 16973828, [2] = 16973829)
    prevRtcTime = 0
    mDataSlab = 0x0003c340
    dataEvt = 0x00000000
    alsLastSample = 0
    psLastSample = 1
    bcRecv = {
        [0] = {
            list = {
                prev = 0x00048290
                next = 0x002109e0
            }
            sensor_type = '\f'
            receive_event = 0x00206a34 (tinysys-scp-RV33_A.elf`alsPsReceiveEvent at alsps.c:908)
        }
    }
}
```

- Dump memory: x/FMT format address

```
(lldb) x/32xw 0x1ef08
0x0001ef08: 0x003a9593 0x093d3533 0x10063667 0x00bb0ab3
0x0001ef18: 0x01340633 0x016ab5b3 0x00b60c33 0x008c0463
0x0001ef28: 0x008c35b3 0x01248633 0x0433955e 0x35b300b6
0x0001ef38: 0x36330096 0x952e00c4 0x00c50bb3 0xd5334d32
0x0001ef48: 0xc11d094b 0x001ad593 0x001af513 0xfe1c05b3
0x0001ef58: 0x001c5c13 0xfe140c33 0x84338005 0xdb93fe1b
0x0001ef68: 0xeab3001b 0x088500a5 0x652144a2 0x09f4a4b3
0x0001ef78: 0xcf63157d 0x053700a8 0x24237fff 0x8d451001
```

- Read CPU register: register read \$REG

```
(lldb) Register to read pc
pc = 0x00000038 tinysys-scp-RV33_A.elf`__divtf3 + 54 at divtf3.c
(lldb) register read
general:
    x0 = 0x00000000 tinysys-scp-RV33_A.elf`vPortInitialiseBlocks at heap_2.c:239
    x1 = 0x0000a32c tinysys-scp-RV33_A.elf`stackDump + 614 at scp_it.c:128
...
    x27 = 0x00000000 tinysys-scp-RV33_A.elf`vPortInitialiseBlocks at heap_2.c:239
    x28 = 0x0000006d tinysys-scp-RV33_A.elf`__divtf3 + 107 at divtf3.c:30
(lldb) read the register mepc
mepc = 0x0000a32c tinysys-scp-RV33_A.elf`stackDump + 614 at scp_it.c:128
```

9.7 Performance Budget and Flight Record

PBFR is a profiling tool that can monitor CPU usage. It can be divided into two sections, the performance budget and the flight record.

The performance budget contains information about each task loading, cache miss, stall, and whole system loading. And the flight record can record the CPU trace during a period.

9.7.1 Performance Budget

This section will introduce the method of using a performance budget to monitor task loading.

- **Enable performance budget:**

Configure and define settings to activate the performance budget.

- Path
 - vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/inc/FreeRTOSConfig.h
 - vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/platform.mk

Table 9-3: Compiler options and definitions

#	Compiler Options and Definitions	Description
1	CFG_PBFR_SUPPORT	PBFR main function control option Enable: Yes Disable: No
2	#define PBFR_SUPPORT_POLLING	Support saving the maximum task load by polling
3	#define PBFR_SUPPORT_POLLING_MS 10	Polling task activity every 10ms
4	#define PBFR_SUPPORT_CACHE_COUNT	Support cache miss rate
5	#define PBFR_SUPPORT_IOSTALL	Support IO stall rate

- **Usage1:**

- Per 10 sec report Performance Budget
- CFG_MONITORTASK_ALWAYS_RUN = yes

- **Usage2:**

- Command: (start/report/end)
 - \$ echo 666 0 1 > sys/class/misc/scp/scpctl
- Enter the command for the 1st time to start recording performance budget information; in a period of time, enter the command for the 2nd time to report the loading info; and then enter the command for the 3rd time to stop.
- For some customized requests, the following three APIs can be added to a specific place for monitoring partial jobs.
 - pbfr_start_loadinfo(0): Start to record loading information
 - pbfr_report_loadinfo(0): Report the loading information
 - pbfr_stop_loadinfo(0): Stop to record loading information

• Log interpretation:

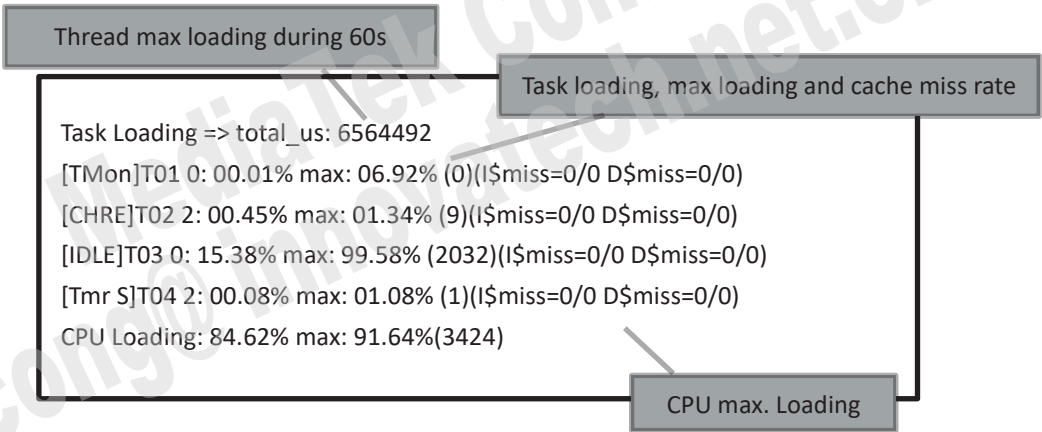


Figure 9-5. Performance budget log example

9.7.2 Flight Record

Flight Record is a debug tool that can record the last events when the system crashes. The following will introduce how to use flight record to monitor CPU traces.

• Enable Flight Record:

Set the configuration and activate definitions to obtain flight records. The configuration in `platform.mk` enables Flight Record, and the definitions in `FreeRTOSConfig.h` specify the events that can be recorded.

- Path
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/platform.mk

Table 9-4. Compiler options

#	Compiler Option	Description
1	CFG_PBFR_SUPPORT	PBFR main function control option Enable: Yes Disable: No

- Path
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/inc/FreeRTOSConfig.h

Table 9-5. Definitions of Flight Record

#	Definitions	Description
1	#define PBFR_SUPPORT_FLIGHT_REC	Support flight record
2	#define PBFR_MAX_REC_EVENTS 32	Support 32 trace events (maximum: 255)
3	#define PBFR_SUPPORT_REC_TASK	Support tracing of task events
4	#define PBFR_SUPPORT_REC_QUEUE	Support queue trace events
5	#define PBFR_SUPPORT_REC_SWTIMER	Support software timer trace events

#	Definitions	Description
6	#define PBFR_SUPPORT_REC_INT	Support tracing events in critical sections
7	#define configTRACE_INT_MASK_TIME_BOUD_NS	Record critical section trace event only if duration exceeds the time bound
8	#define PBFR_SUPPORT_REC_ISR	Support interrupt service routine trace events
9	#define PBFR_SUPPORT_REC_OSTICK	Support OS tick trace events

If the build failed out of memory size, change the setting in **Setting.ini**.

- Log: "SCP: {XXX}(size_A>size_B) is out of memory limitation", change the size of XXX (size_B) larger than size_A.

- Path

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/Setting.ini

- Usage:

- Copy **tinysys-scp-RV55_A.elf** and **SCP coredump** to the analysis folder.
 - Analysis folder: alps/prebuilts/clang/md32rv/linux-x86/lldb_v3
- Parsing data with the core dump analyzing tool to get the SCP_debug.txt and flrec.json.**
- If successful, it will show the following log.

```
(lldb)command script import
/proj/mtk11261/misc/coredump/scp_coredump_cmdline/bin/coredump/flrec.py
(lldb) flrec
The Flight Recorder is working ...
Flight record output completed.
(lldb) command script import
/proj/mtk11261/misc/coredump/scp_coredump_cmdline/bin/coredump/fflrec.py
(lldb) fflrec
The flight recorder is working...
('xTickCount:', 82801L)
len=1000 status=00000003 RP=123 WP=123 last=82809147582
virtual ns = 82320378580
Flight record output completed.
(lldb)
```

- Load flrec.json in chrome://tracing/ to get the GUI result.

- Log interpretation:**

- Console mode:

Console mode is present in SCP_debug.txt; it shows the recorded events before the system crashed and displays events classified by the type. Record[X] indicates that this event is the Xth closest to last event.

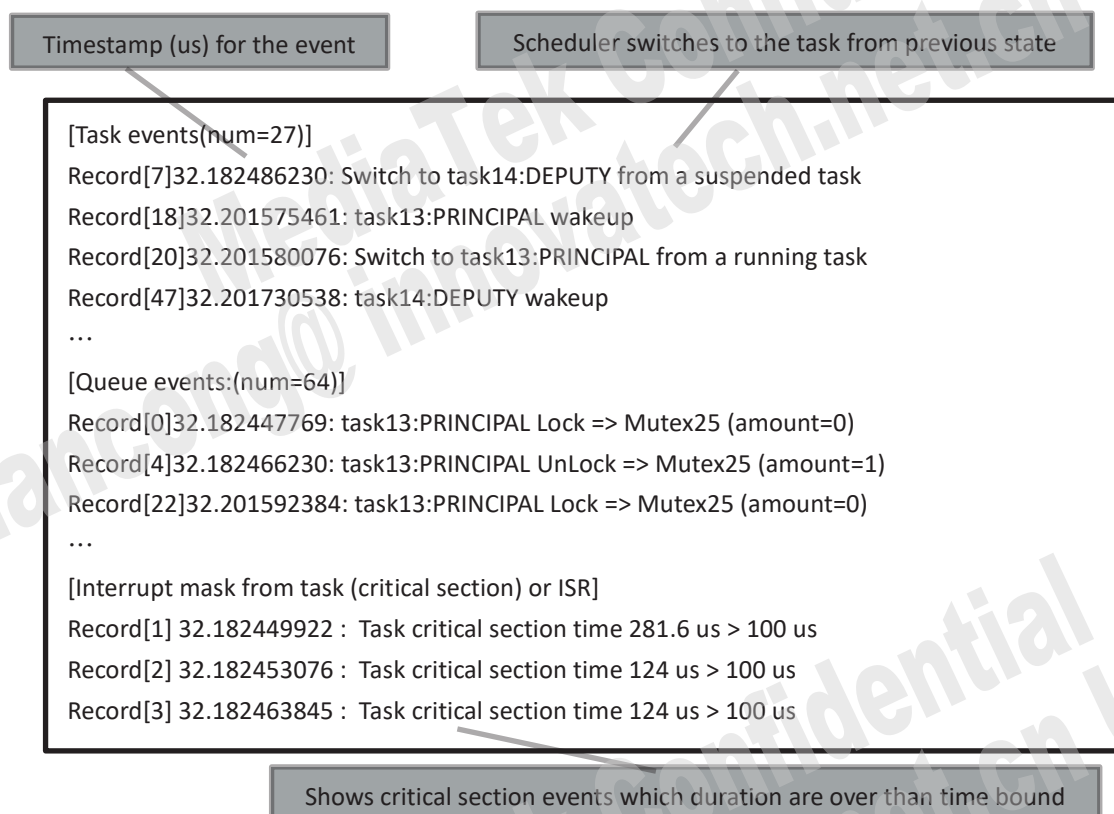


Figure 9-6. Flight record console mode example

- GUI mode:
 - Load step
 1. GUI Website: `chrome://tracing/`
 2. Load JSON file: `flrec.json` (created in the analysis folder)
 - Instruction of use

Use keyboard **W/S** to zoom in/out, **A/D** to move left/right on the Website, or switch button to change mouse mode.

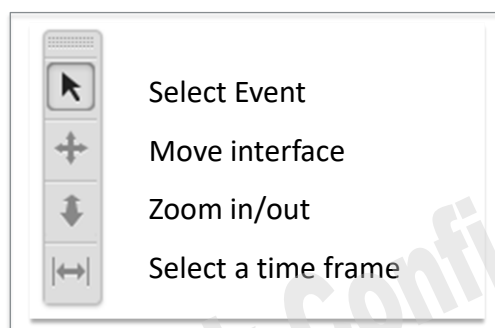


Figure 9-7. Button instruction

Overall

The GUI shows the last events flow, it records six types of events, including task, queue event, software timer, critical section, ISR event, and OS tick.

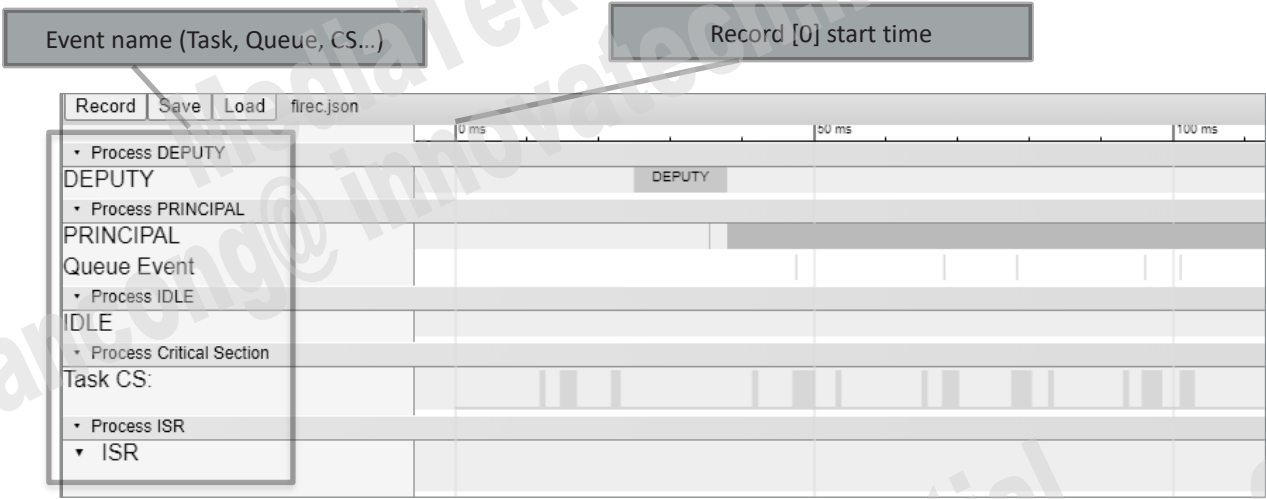


Figure 9-8. Flight record GUI mode interface

The message on the bottom shows the event's start time and duration.



Figure 9-9. Detail information

Task

The task event shows the task's wakeup time and execution time.

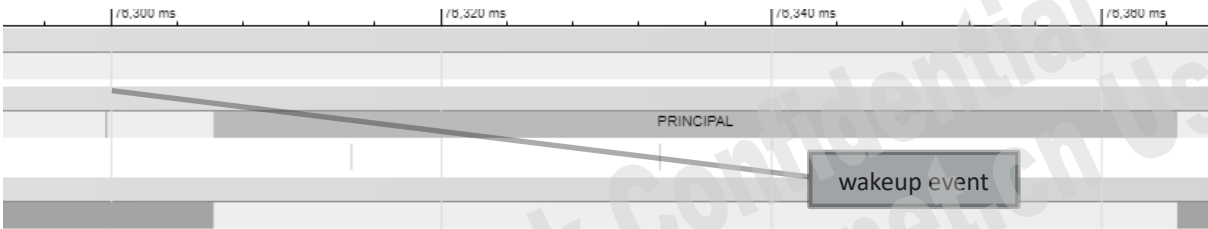


Figure 9-10. Task wakeup event

• Queue

The queue event is presented below the task which executed the event and clicking event can see the type of queue shown in the information bar.

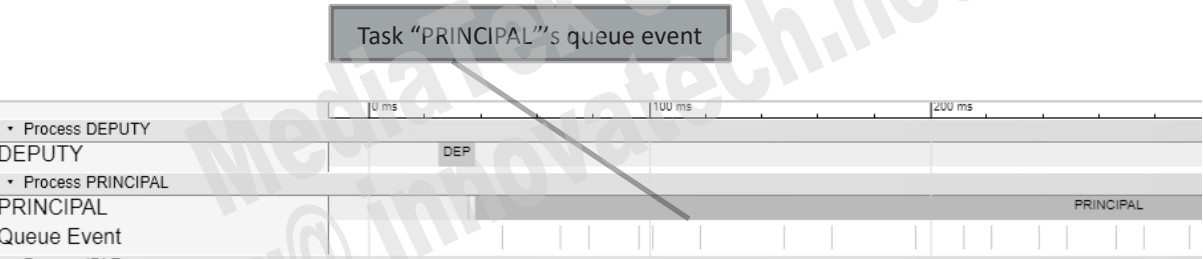


Figure 9-11. Example of queue event



Figure 9-12. Detail information of queue event

• Software timer

SW timer shows the execution time of the callback function.

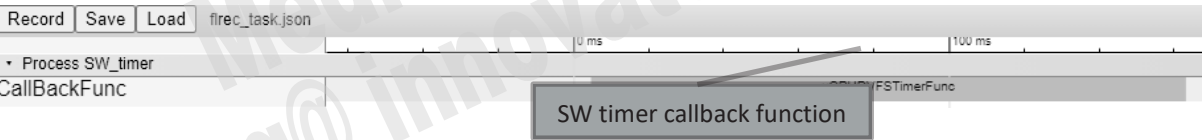


Figure 9-13. Example of SW timer

• Critical Section

By moving upward through the corresponding critical section, you can find the tasks running in the critical section.

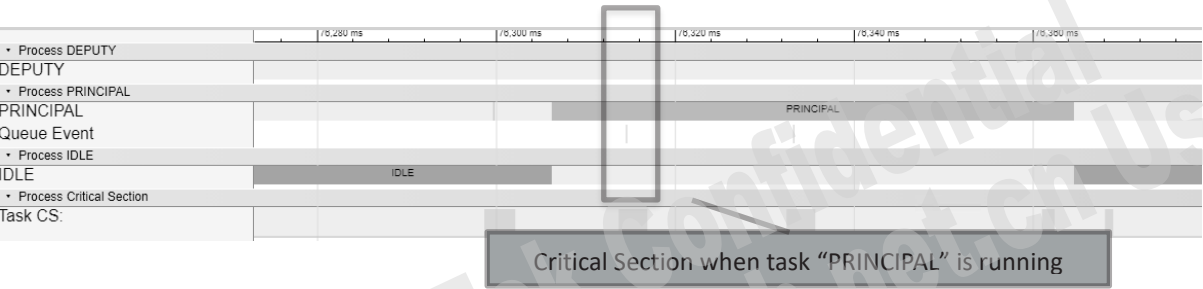


Figure 9-14. Example of critical section

• **ISR**

The execution time of the ISR and the ISR number are shown in [Figure 9-15](#).

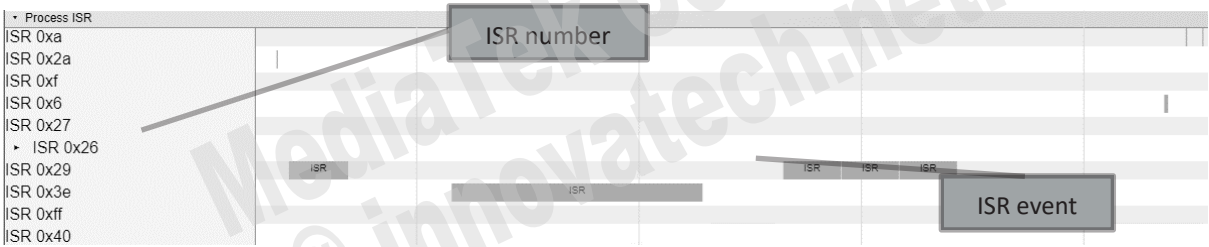


Figure 9-15. Example of ISR

• **OS Tick**

OS Tick shows the time when the OS timer ticks.

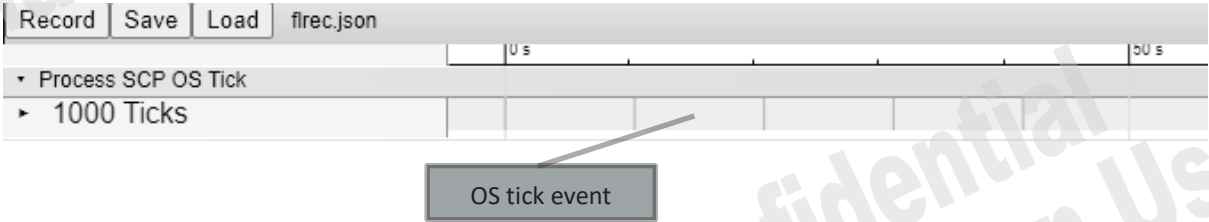


Figure 9-16. Example of OS Tick

10 Address Sanitizer

10.1 ASAN Usage

Address Sanitizer (ASAN) is an address sanity checker that finds out-of-bounds accesses to heap, stack, and global objects, as well as use-after-free bugs.

- **Enable ASAN**
 - Set the configurations and turn on definitions to enable ASAN.
 - Path
 - vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/\$Project/project.mk
 - vendor/mediatek/proprietary/tinysys/scp/build/config.mk

Table 10-1. Compiler options and definitions

#	Compiler Options and Definitions	Description
1	CFG_ASAN_SUPPORT	Enable: Yes Disable: No
2	-mllvm -asan-stack	Check local variable Enable: 1 Disable: 0
3	-mllvm -asan-global	Check global variable Enable: 1 Disable: 0
4	-mllvm -asan-memintrin	Check memset/memcpy Enable: 1 Disable: 0

- **Exception**

Table 10-2. Exception type

#	Exception Type	Description
1	ASAN_STACK_LEFT	Local variable underflow
2	ASAN_STACK_MID	Local variable underflow/overflow
3	ASAN_STACK_RIGHT	Local variable overflow
4	ASAN_STACK_PARTIAL	Local variable overflow
5	ASAN_GLOBAL_REDZONE	Global variable overflow
6	ASAN_HEAP_USE_AFTER_FREE	Heap use after free

• Issue handling - Stack underflow

```
[14.504] (0) [0] KASAN detect stack-out-of-bounds at PC:0x15f52
[14.505] (0) [0] write 1Byte at [0x3c298] -> shadow [0x207853]
[14.506] (0) [0] Shadow bytes around the buggy address:
[14.507] (0) [0] 0x00207830: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.508] (0) [0] 0x00207840: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.509] (0) [0] =>0x00207850: f1 f1 04 [f2] 00 02 f3 f3 00 00 00 00 00 00 00 00
[14.510] (0) [0] 0x00207860: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.512] (0) [0] 0x00207870: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.513] (0) [0] [ASSERT] task: Main, file: ../common/middleware/sanitizer/asan/asan_helper_v1.c, line: 144
```

```
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Global redzone: f9
```

```
00015f42 <asan_underflow_left>:
; asan_underflow_left():
; /mfs/mtk/mt1157/mtk26942/casws/alps-mp-t0-mp1-of-p8_b/vendor/
; {
15f42: 9005      push16.qw.m0
15f44: 717d      c.addi16sp sp, -16
; /mfs/mtk/mt1157/mtk26942/casws/alps-mp-t0-mp1-of-p8_b/vendor/
; ptr[-1] = 0xAA;
15f46: fff50493  addi    s1, a0, -1 // 0xffff [12bitSE]
15f4a: 842a      c.mv    s0, a0
15f4c: 8526      c.mv    a0, s1
15f4e: 8adf40ef  jal    -46932 <__asan_store1>
15f52: 0aa00513  li     a0, 170
```

```
attribute__((noinline)) void asan_underflow_left(char *ptr)
{
    ptr[-1] = 0xAA;
    printf("%s: underflow write %p\n", func, &ptr[-1]);
}
```

• Issue handling - Global variable overflow

```
[8.490] (0) [0] KASAN detect global-out-of-bounds at PC:0x15f64
[8.491] (0) [0] read 4Byte at [0x60288] -> shadow [0x20c051]
[8.492] (0) [0] Shadow bytes around the buggy address:
[8.493] (0) [0] 0x0020c030: f9 f9 f9 f9 00 f9 f9 f9 f9 f9 f9 04 f9 f9 f9
[8.494] (0) [0] 0x0020c040: f9 f9 f9 f9 00 f9 f9 f9 f9 f9 f9 00 00 00 00
[8.495] (0) [0] =>0x0020c050: 00 [f9] f9 f9 f9 f9 04 f9 f9 f9 f9 f9 f9
[8.496] (0) [0] 0x0020c060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[8.498] (0) [0] 0x0020c070: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[8.499] (0) [0] [ASSERT] task: Main, file: ../common/middleware/sanitizer/asan/asan_helper_v1.c, line: 144
```

```
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Global redzone: f9
```

```
; /mfs/mtk/mt1157/mtk26942/casws/alps-mp-t0-mp1-of-p8_b/vendor/mediatek
; test = ptr[10];
15f52: 00060537  lui    a0, 96 // 0x60000
15f56: 26050413  addi    s0, a0, 608 // 0x260 (132: 0x60260)
15f5a: 02840493  addi    s1, s0, 40 // 0x28
15f5e: 8526      c.mv    a0, s1
15f60: 80bf40ef  jal    -47094 <__asan_load4>
15f64: 854a      c.mv    a0, s2
```

```
attribute__((noinline)) void global_overflow(void)
{
    volatile uint32_t *ptr;
    volatile uint32_t test;
    ptr = &globalArr[0];
    test = ptr[10];
}
```


- **Issue handling - Heap use after free**

```
[9.230] (1) [0] KASAN detect heap use-after-free at [PC:0x40c2a]
[9.231] (1) [0] read 4Byte at [0x271b90] -> shadow [0x54e372]
[9.232] (1) [0] Shadow bytes around the buggy address:
[9.232] (1) [0] 0x0054e350: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[9.233] (1) [0] 0x0054e360: 00 00 00 00 00 00 00 00 00 00 00 00 fd fd fd fd
[9.234] (1) [0] =>0x0054e370: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
[9.235] (1) [0] 0x0054e380: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
[9.236] (1) [0] 0x0054e390: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
[9.236] (1) [0] [ASSERT] task: Main, file: ../common/middleware/sanitizer/asan/asan helper v1.c, line: 135
```

Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Global redzone: f9
Heap use after free: fD

```

/mfs/mtkslt1157/mtk26942/casws/alps-mp-t0_mp1-of_p
if(ptr[5] == 0xAA)
40c22: 01440513    addi    a0, s0, 20 // 0x14
40c26: 873cb0ef    jal    -214926 <_asan_load4>
40c2a: 4848      c.lw    a0, 20(s0)
40c2c: 0aa00593    li     a1, 170
40c30: 00b51863    bne    a0, a1, 16 <.LBB9_2>

```

```
uint32_t *dynamicArr;  
attribute__((noinline)) void asan_use_after_free(void)  
{  
    uint32_t *ptr;  
    dynamicArr = pvPortMalloc(10*sizeof(uint32_t));  
    vPortFree(dynamicArr);  
    ptr = dynamicArr;  
    if(ptr[5] == 0xAA)
```

10.2 How to Move Code to DRAM

After you enable ASAN, the code size will increase about 1.7x. Therefore, the SRAM size might be insufficient. You can move some feature or function to DRAM to reduce SRAM usage.

- **Use attribute**

To specify a function/variable in the DRAM region, use cache macros defined in `cache` internal.

Note: DRAM_REGION_VARIABLE operates exclusively on global/static variables as local variables utilize the stack for storage. By default, for performance optimization, the stack is retained in SRAM.

```
#define DRAM_REGION_VARIABLE __attribute__((section(".dram_region_variable")))
#define DRAM_REGION_FUNCTION __attribute__((section(".dram_region_func")))
```

- **Use linker script**

The linker script describes how the objects' sections should be mapped in the output binary.

An easy method is provided for locating objects to the DRAM region.

The following provide a template header and linker script to locate all sections in a C file in the DRAM region.

1. Check `cache_ld.h` (`@project/RV55_A/$platform/$project`). Here you can see lots of predefine macros with style `CACHE_***_TEXT` & `CACHE_***_DATA`. You can add one and name it by yourself. The string after name is the expected C file path.

For example: (The “?” is wildcards compare)

```
#define CACHE_BARO_TEXT \
    ?*/middleware/contexthub/MEMS_Driver/barometer/?*.o(.text*) \
#define CACHE_BARO_DATA \
    ?*/middleware/contexthub/MEMS_Driver/barometer/?*.o(.rodata* .data* .bss*) \
```

2. Check `link.ld`. It should have the response `CACHE_***_TEXT` & `CACHE_***_DATA`.
You can add the string after `CACHE_BARO_TEXT` / `CACHE_BARO_DATA`.

For example:

```
dram_region __dram_start : AT(__dram_start) {  
#ifdef CACHE_BARO_TEXT  
    CACHE_BARO_TEXT  
#endif  
  
    *(.dram_region_func)  
    *(.dram_region_ro)  
  
#ifdef CACHE_BARO_DATA  
    CACHE_BARO_DATA  
#endif  
}
```

11 Appendix

11.1 Common Issues and How to Fix

11.1.1 Malloc Fail

Because the SRAM size is limited and small, malloc pool size is tailored to the feature, malloc fail is common when a new feature is involved without enlarging the pool size.

Note:

- When malloc fail happens, a fail message will be shown:

```
[2.232] (0) malloc fail
```

```
[2.232] (0) [ASSERT] task: CHRE
```

- This message shows which task malloc fail and then will follow the coredump log.

- How to Fix

1. Modify the `scp/project/RV55_A/mt8189/platform/platform.mk`, enlarge the heap size, e.g., 80×1024

```
ifeq ($(CFG_CHRE_SUPPORT),yes)
$(eval TOTAL_HEAP_SIZE=$(shell echo $$(( $(TOTAL_HEAP_SIZE) + (80 * 1024) ))) ) • change 40 to 80
endif
```

2. Modify the `scp/drivers/common/scpctl/scp_scpcctl.c`, force enable Task monitor.

```
void scpctl_init(void)
{
...
    else { /* monitor task is in suspended state */
        scpctl.stat = SCPCTL_STAT_INACTIVE;
        scpctl.op = SCPCTL_OP_INACTIVE;
        //vTaskSuspend(xMonitorTask[id]); • about line: 244, mark it
    }
...
}
```

3. Check the remaining heap size and compute the required heap size.

```
[210.009] (0) Heap:free/total:40946/108544
```

4. Correct the heap size: the remaining heap size is **40946**, $40946/1024 = 39.98$, the minimum heap size is 41, and setting to 43 or 44 is safer. Please restore `scp/drivers/common/scpctl/scp_scpcctl.c` change after the issue is fixed.

```
ifeq ($(CFG_CHRE_SUPPORT),yes)
$(eval TOTAL_HEAP_SIZE=$(shell echo $$(( $(TOTAL_HEAP_SIZE) + (43 * 1024) ))) )
endif
```

11.1.2 Unaligned Access

The SCP MDSP-RV33 processor does not have hardware to handle unaligned access; the implementation is done in software. When unaligned access happens, the processor will raise exceptions and do a software solution in the handler. Therefore, the performance cost is significantly reduced.

Note:

- When unaligned access happens, a warning message will be shown.
"Warning: MISALIGNED LOAD, pc:0x00001234, addr:0x00042232"
- This message shows the PC with problems and address the processor wants to access. The system may be busy to print this message when unaligned access happens continuously and may cause timeout assert.
- Example and how to fix

The majority of unaligned access occurs because of the packed qualifier applied to the structure. This issue could be avoided by removing it.

Example:

```
struct pack_struct {
    unit32_t size;
    uint32_t crc;
    uint8_t type;
} __attribute__((packed));
```

When a structure array is declared, for example, `pack_struct st_array[10]`, the access of structure member will be unaligned.

Another case is accessing a character array with an integer pointer. This could be avoided by adding `__attribute__((aligned(4)))`. The compiler does not guarantee `&char_array[0]` is 4 byte alignment.

Example:

```
uint8_t char_array[64];
test_value = *(uint32_t *)(&char_array[0])
```

11.2 Code Size Limitation

The total SRAM size of MT8391 is 384KB; and the actual SCP SRAM size can be determined by checking the symbol `_end`.

The remaining SRAM size:

MT8391: `0x300000 - _end`

In MT8391, you can set SRAM region LENGTH to 384KB in the `project/RV55_A/mt8189/platform/link.ld.c`

- Example:

SRAM: `ORIGIN = 0x00000000, LENGTH = 0x00200000.`

. When size exceeds 384KB, this will cause build failure.

The code size check tool will be executed in every build to provide output details. Refer to the following table, where vertical represents the item name and horizontal indicates the code size of each item. For instance, the total size of CHRE is 62985 (Sum) bytes, with the `.text` section occupying 35598 bytes of the total size.

	. = ALIGN (...sync		.text	Sum
C-lib	0	...	0	0
CHRE	0	...	35598	62985
DSP	0	...	0	0
DVFS	0	...	7574	8606
Heap	0	...	334	82275
Peripheral	0	...	18064	26981
Platform	256	...	66300	313481
RTOS	0	...	13776	14536
Sensor	0	...	0	0
VOW	0	...	0	0

11.2.1 Code Size Tool Usage

- The tool, `memoryReport.py`, is a script used to limit code size at build time. If the code size exceeds your settings, it will cause build errors.
 - Path
`vendor/mediatek/proprietary/tinysys/common/tools/memoryReport.py`
- Setting configuration file at the following path:
 - Path
`vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/Setting.ini`

Configuration file format (setting.ini)

```
-----  
[TinySys-SCP]  
$File_Name: $Main_feature: $Sub_feature  
[SCP-MT6985]  
$Main_feature : Max_code_size  
$Sub_feature : Max_code_size  
-----  
* File_name: Full file path or Partial file path (Ex: middleware/contexthub/perf)  
* Main feature, (Ex: Sensor, Audio), the main feature that this file belongs to  
* Sub feature, (Ex: gyro, pedometer), the sub feature that this file belongs to  
* Main_feature/Sub_feature (after SCP-MT6985):  
  - Main or Sub feature maximum size limit
```

- Memory check fail
For example:

```
SCP: I2C(3958>110) is out of memory limitation  
SCP: SPI(6316>1100) is out of memory limitation  
make: *** [tinysys_out/RV55_A/scp/tinysys-scp-RV55_A.elf] Error 13
```

11.3 Scp_Region_Info Structure

The struct `scp_region_info_st` is a pointer to a fixed address at SCP SRAM. It's used to pass parameters from bootloader to SCP before the IPI ready. Remember to sync bootloader (lk2), kernel (kernel-6.1) and SCP repos code if a new member is added. SCP may fail to boot if the structure is not synchronized between repositories.

- LK header:
vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/mediatek/mt8189/common/scp/scp_plat_priv.h
- Kernel header:
kernel/kernel_device_modules-6.1/drivers/misc/mediatek/scp/rv/scp_helper.h
- SCP header:
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/inc/main.h
- SCP boot:
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189 /platform/boot55.S

The definition of `scp_region_info` is also present in `boot55.S`.

For example, if a new member called test ID is added in structure `scp_region_info`, the default value must be added in `boot55.S`.

```
scp_region_info:
.long 0x00000000 /* 0x04 ap_loader_start */
.long 0x00000000 /* 0x08 ap_loader_size */
...
.long 0x00100000 /* 0x34 regdump size */
.long 0x00000000 /* 0x38 param start address */
.long 0x00000000 /* 0x3c ap_params_start */
.long 0x12345678 /* 0x40 a test ID */
```

Access `scp_region_info_st` in SCP side, then assign it to `NULLPTR_BASE`, for example:

```
void scpctl_init(void)
{
    int ret = 0;
#ifdef CFG_NULLPTR_TRAP
    struct scp_region_info_st region_info = *(struct scp_region_info_st *) (NULLPTR_BASE +
0x4);
#endif
}
```

11.4 SCP Recovery

The Linux SCP driver will do SCP recovery when SCP crashes or does not respond. After the recovery process, the SCP will be back to normal state. But please note:

- When SCP is reset, SCP driver will clear the contents of SCP SRAM/DRAM, reset the RV55 processor in ATF, and re-execute the boot process. (Clearing all program text, bss, and data segments)
- SCP recovery does reset only processor **but not** peripherals (e.g., sensors, I2C module and other devices), so peripheral drivers should do self-reset at initial stage if necessary.

- Developers of Linux SCP driver API **MUST** follow “Reset notify flow” in Section 11.4.2 and ensure there are not any communication between Linux driver and SCP during the recovery.
- The Linux SCP driver will re-initialize when SCP reboots. Drivers that relate to the SCP driver **MUST** ensure the reboot flow will not affect their functions.

11.4.1 Recovery Behavior

The recovery starts when the SCP is abnormal or unresponsive for a period of time. The SCP kernel driver and SCP will enter an out of service state. During this period, the SCP kernel driver will reset SCP and send the SCP_EVENT_STOP to all registered notify chain drivers. After SCP is back to normal, the SCP kernel driver will send SCP_EVENT_READY to all the drivers.

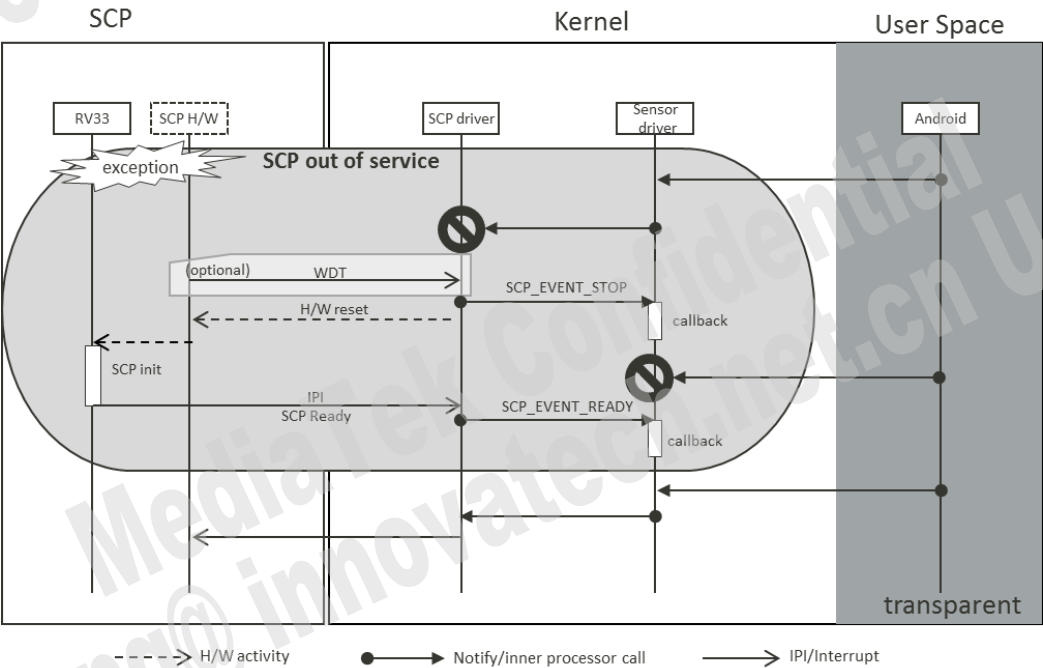


Figure 11-1. SCP recovery behavior

11.4.2 Recovery Notify Flow

Developers of the Linux SCP driver **MUST** follow this SCP EVENT notify chain to ensure SCP recovery is effective.

- **API:** receives notifications and invokes registered callback functions

API	<code>void scp_A_register_notify(struct notifier_block *nb)</code>
Description	To register the callback function of the notify chain
Parameters	• nb: Callback function of the notify chain
Callback parameters	• SCP_EVENT_READY: When the initial SCP is done, start tasks corresponding to the SCP. • SCP_EVENT_STOP: When SCP is going to be reset, stop tasks corresponding to SCP.

Note:

- All callback functions may be called multiple times and **MUST NOT** be blocked.

- **Example**

1. Please include `<linux/notifier.h>`, `<mach/scp_helper.h>`
2. Call `scp_register_notify()` with argument: `SCP_EVENT_READY` or `SCP_EVENT_STOP`

```
static void task_start(void) {
    // start tasks
}
static void task_stop(void) {
    // stop tasks
}
static int app_event(struct notifier_block *this, unsigned long event, void *ptr) {
    switch (event) {
        case SCP_EVENT_READY:
            task_start();
            break;
        case SCP_EVENT_STOP:
            task_stop();
            break;
    }
    return NOTIFY_DONE;
}
static struct notifier_block app_notifier = {
    .notifier_call = app_event,
};
static int __init scp_app_init(void) {
    if (scp_is_ready()) {
        task_start();
    }
    scp_register_notify(&app_notifier);
}
```

Note:

- **MUST** register notification chain because the `scp_ipi_send()` may return error during recovery.
- An error handling flow **MUST** be applied:
 - Stop calling `scp_ipi_send()` right after receiving `SCP_EVENT_READY`
 - Resume after receiving `SCP_EVENT_READY`

12 Q&A List from Users

12.1 How to Enlarge DRAM Region Code

By default, 1MB of DRAM is reserved for storing DRAM code. In an extreme case where more space is needed to store code, the modification method is provided.

1. Modify LK, enlarge 0x100000

```
vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/mediatek/common/scp/RV/scp_plat_priv.h
#ifdef MTK_MINIMUM_SCP_DRAM_SIZE
#define SCP_DRAM_IMG_SIZE      0x080000    // 0.5MB dram image
#else
#define SCP_DRAM_IMG_SIZE      0x100000    // 1.0MB dram image
#endif
```

2. Modify SCP, enlarge 0x00100000

```
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt8189/platform/link.ld.c
MEMORY {
    ... ..
    dram      :    ORIGIN = 0x00200000, LENGTH = 0x00100000
    ... ..
}
```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.